Evaluating Post-Quantum Key Exchange in WireGuard

A Comparative Study of ML-KEM and HQC

Ludvig Järvi

supervised by

Karl Andersson

John Lindström

Jonas Olson

Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden

May 11, 2026

Acknowledgements

Abstract

Keywords: WireGuard, VPN, Post-Quantum, PQ, Post-Quantum Cryptography, PQC, ML-KEM, HQC, Key Encapsulation Mechanism, KEM, Hybrid Cryptography

Contents

Glossary

DH Diffie-Hellman.

FS Forward Secrecy.

HQC Hamming Quasi-Cyclic.

KEM Key Encapsulation Mechanism.

ML-KEM Module-Lattice-Based Key Encapsulation Mechanism.

NIST the National Institute of Standards and Technology.

PKE Public-Key Encryption.

PQ Post-Quantum.

PQC Post-Quantum Cryptography.

PQFS Post-Quantum Forward Secrecy.

PSK Pre-Shared Key.

QKD Quantum Key Distribution.

VPN Virtual Private Network.

1 Introduction

Cryptographic protocols such as Transport Layer Security (TLS), Secure Shell (SSH), and Virtual Private Networks (VPNs) rely on asymmetric cryptography to securely establish shared secrets between previously untrusted parties. These cryptographic systems are based on the assumption that specific mathematical problems, notably integer factorization and discrete logarithms, are too hard for classical computers to solve efficiently.

However, this assumption may not hold in the long term. Rapid advances in quantum computing research suggest that large-scale, fault-tolerant quantum computers may become viable within the coming decades [1]. These are computers capable of leveraging the laws of quantum mechanics to perform quantum algorithms that achieves significant computational speedups for specific classes of problems in comparison to their classical counterparts. Such machines would be able to fundamentally break the asymmetric cryptography primitives used today.

Two quantum algorithms stand out in this cryptographic context. Peter Shor's algorithm can efficiently factor large integers and compute discrete logarithms [2], and Lov Grover's algorithm [3], which provides a quadratic speedup to brute-force key searches. The former directly breaks the public-key primitives underlying RSA, Diffie-Hellman and elliptic-curve cryptography, while the latter reduces the effective security level of symmetric ciphers and hash functions by half.

Once quantum computers reach this critical scale, they will likely be capable of decrypting nearly all historical and contemporary communications secured with current public-key systems. Moreover, adversaries can already capture encrypted traffic today with the intent of decrypting it later when quantum capabilities become available, a strategy commonly referred to as "harvest now, decrypt later" [4]. This threat makes it essential to begin the transition toward quantum-resistant cryptography as soon as possible, since deploying new cryptographic standards across the Internet ecosystem will take many years.

To stay ahead of this threat, the cryptographic community is actively developing Post-Quantum Cryptography (PQC) algorithms. These are new types of "quantum-resistant" mechanisms designed to be secure against both today's computers and tomorrow's quantum machines. The algorithms are based on mathematical problems believed to remain hard for quantum computers, such as those derived from lattices, error-correcting codes, or hash functions. The National Institute of Standards and Technology (NIST) initiated, back in 2016, a global effort to identify and standardize these new methods [5], resulting in several promising candidates.

After several years of public evaluation, the NIST Post-Quantum Cryptography Standardization Process selected and standardized ML-KEM [6], a lattice-based Key-Encapsulation Mechanism (KEM), and is currently in the process of standardizing Hamming Quasi-Cyclic (HQC) [7], a code-based KEM. These schemes are considered primary building blocks for securing Internet communications in the quantum era.

While the theoretical security of these algorithms has been extensively studied, their practical deployment within real-world communication protocols remains an active research challenge. Post-quantum schemes often have larger key sizes, ciphertexts,

and computational overhead compared to classical cryptographic methods. Such differences can significantly affect performance-sensitive protocols that prioritize efficiency.

One such protocol is WireGuard [8], a modern VPN protocol designed for minimalism, simplicity, and high throughput. WireGuard currently relies on elliptic-curve cryptography, specifically Curve25519, as part of its authenticated key exchange mechanism. While this design offers excellent efficiency on classical hardware, elliptic-curve cryptography is vulnerable to quantum attacks due to its reliance on the discrete logarithm problem.

Integrating PQC into WireGuard therefore presents several unique challenges related to efficiency, compatibility, and code maintainability. These include managing increased message sizes, maintaining low handshake latency, and preserving the protocol's minimalist design philosophy. Existing solutions, such as Rosenpass and PQ-WireGuard prototypes, have demonstrated the feasibility of quantum-resistant VPN connections through external or hybrid key exchanges [9][10][11].

Yet, few studies have systematically compared the performance characteristics of NIST-approved algorithms when integrated into WireGuard's architecture.

This thesis addresses that gap by designing, implementing, and evaluating a hybrid WireGuard handshake that supports two NIST-selected Key-Encapsulation Mechanisms: ML-KEM and Hamming Quasi-Cyclic. The implementation enables an empirical comparison of their latency, computational overhead, memory usage, and message size, and contrasts the results against existing post-quantum WireGuard implementations. By quantifying these trade-offs, the work aims to provide practical insights into the real-world viability of post-quantum VPNs and contribute to the broader effort of securing Internet communications against future quantum threats.

1.1 Problem statement and research questions

How does NIST-approved Post-Quantum Key-Encapsulation Mechanisms, specifically ML-KEM and HQC, affect the performance, resource usage, and practical deployability of a post-quantum secure WireGuard handshake compared to existing approaches?

RQ1. How do NIST-selected post-quantum key encapsulation mechanisms, specifically ML-KEM and HQC, affect the performance of a WireGuard handshake?

RQ2. What impact do ML-KEM and HQC have on resource usage in WireGuard, particularly in terms of message size and storage requirements?

RQ3. To what extent can ML-KEM and HQC be integrated into WireGuard without violating its core design principles, such as low latency and small handshake size?

RQ4. How do hybrid cryptographic approaches (combining classical Diffie-Hellman with PQC) compare to purely post-quantum approaches in terms of security, performance, and deployability?

RQ5. How does a hybrid WireGuard implementation using ML-KEM and HQC compare to existing post-quantum solutions in terms of performance and practical deployment?

RQ6. What trade-offs exist between security level and ciphertext size when selecting PQC algorithms for use in WireGuard?

1.2 Scope and Limitations

This thesis covers the design, implementation, and empirical evaluation of post-quantum KEM integration into WireGuard's handshake, using ML-KEM and HQC at NIST security levels 3 and 5. The implementation contains both a hybrid mode, which augments the classical handshake with an additional KEM operation, and a pure-KEM mode, which replaces the classical exchange entirely, are implemented and benchmarked.

Several limitations apply to this work. The implementation is built using `wireguard-go`, the userspace Go implementation of WireGuard, and does not modify the Linux kernel module. Performance numbers from the kernel implementation would be better due to the absence of TUN interface overhead, so the results here should not be taken as representative of a future kernel-level deployment.

All benchmarks were conducted on a single hardware platform: two virtual machines running on the same KVM hypervisor. No measurements were taken on physical server hardware, ARM-based processors, or mobile devices.

The benchmark environment uses a virtual network on a single physical host, which eliminates packet loss and real-network jitter. In particular, large handshake messages that exceed the standard MTU are fragmented and reassembled entirely in software, without the reliability problems that IP fragmentation causes on real network paths. Thus, the HQC results in this thesis are measured under more favorable conditions than a real deployment would provide.

The security analysis of the hybrid and pure-KEM handshake constructions is informal. No formal verification or symbolic proof is provided, and the security arguments in this thesis are based on design reasoning rather than machine-checked proofs. This is consistent with the scope of the related work that this thesis builds on, but it means the security claims are weaker than those of formally analyzed designs.

1.3 Thesis Structure

The remainder of this thesis is structured as follows.

Section 2 provides the necessary background. It begins with classical cryptography, covering classical public-key primitives such as RSA, Diffie-Hellman, and elliptic-curve cryptography, along with key concepts such as key exchange and key encapsulation mechanisms. It then introduces the quantum threat, including a high-level overview of quantum computing and the algorithms that break classical cryptography: Shor's and Grover's algorithms, as well as the "harvest now, decrypt later" risk model. Finally, it introduces post-quantum cryptography, covering the main algorithm families with a focus on ML-KEM and HQC, and discusses hybrid cryptographic approaches and the practical challenges of deploying PQC in real protocols.

Section 3 covers the WireGuard protocol. It describes its design principles, the Noise

Protocol Framework, the IKpsk2 handshake pattern, the cryptographic primitives in use, the full handshake flow, and the Go implementation. It also discusses the specific challenges that arise when integrating post-quantum algorithms into WireGuard.

Section 4 discusses the related work on post-quantum WireGuard, covering the original PQ-WireGuard by Hülsing et al., the Rosenpass sidecar approach, and the recent Rebar redesign using reinforced KEMs by Hashimoto et al.

Section 5 describes concurrent work, covering the ongoing NIST standardization of HQC and the deployment of hybrid ML-KEM in TLS 1.3.

Section 6 describes the research method. It covers the literature study, protocol analysis, design decisions, implementation of both the hybrid and pure-KEM handshake variants, measurement methodology, and benchmarking setup. It concludes with a discussion of reliability, validity, and generalizability.

Section 7 presents the results of the benchmark: handshake latency, message sizes, and storage requirements for all nine evaluated variants, compares them against the related work, and describes how they answer the research questions.

Section 8 outlines the contributions of the thesis to literature, practice, and management, examines the MTU constraint in detail, addresses the requirements of full post-quantum authentication, and compares the implementation with PQ-WireGuard, Rosenpass, and Rebar. It also considers the ethical and sustainability implications of the transition to post-quantum cryptography, and aims to direct future research.

Section 9 concludes the thesis by summarising the main findings and reflecting on the broader implications for post-quantum VPN deployment.

2 Background

2.1 Asymmetric Cryptography

Asymmetric cryptography, also referred to as public-key cryptography, addresses the fundamental problem of symmetric cryptography in secure communications: how can two previously unknown parties establish a secure channel without first meeting up in person to exchange a secret key.

In order to solve this problem, asymmetric cryptography uses two mathematically related keys: a private key and a public key. The public key can be distributed freely, while the private key must remain secret. Messages encrypted with a public key can then only be decrypted using its corresponding private key, enabling secure communications without the need for a predetermined shared secret.

To demonstrate this concept, imagine two people: Alice and Bob. Alice wants to send a secret message to Bob. To achieve this, they do the following:

1. Bob sends a box and an open padlock, to which only Bob has the key, to Alice.
2. Alice places her message in the box, locks it with the padlock, and sends it back to Bob.
3. Bob uses his key to unlock the box and is able to read Alice's message.

Even if some adversary were to intercept the box on its way back to Bob, they would be unable to open it and read Alice's message since they don't have Bob's key.

This analogy describes the core idea of asymmetric cryptography: separating the methods of encryption and decryption. Here the padlock represents the public key, and Bob's key the private key. He can send copies of his padlock to anyone who wants to send him a secret message, knowing only he can open them.

In practice, these systems use specific mathematical problems known as trapdoor functions [12]. These are problems that are easy to compute in one direction, but computationally infeasible to reverse without hidden information (the private key).

Asymmetric cryptography is not used directly to encrypt large amounts of data due to its computational overhead. Instead, it is commonly used during the initial handshake phase of a protocol to establish a shared symmetric key. Once this shared secret has been established, symmetric encryption algorithms can be used to protect the bulk of the communication efficiently.

2.1.1 RSA and Diffie-Hellman

The first generation of widely adopted asymmetric systems relied on two primary mathematical hurdles. The RSA (Rivest-Shamir-Adleman) algorithm, introduced in 1977 by Ron Rivest, Adi Shamir, and Leonard Adleman [13], built on the mathematical difficulty of factoring large composite integers. While multiplying two large prime numbers is computationally trivial, determining the original factors from their product is believed to be infeasible for sufficiently large numbers using classical algorithms. This asymmetry forms the mathematical foundation of the RSA algorithm.

The other fundamental primitive introduced to public-key cryptography was the Diffie-Hellman (DH) key exchange, introduced in 1976 by Whitfield Diffie and Martin Hellman [14]. Rather than encrypting messages directly, Diffie-Hellman introduced the concept of a secure key exchange, enabling two parties to establish a shared secret over an insecure channel without ever transmitting the secret itself. As illustrated in Figure 1, each party generates a private key and exchanges a corresponding public key derived from it. Both parties can then independently compute the same shared secret from the other's public key and their own private key. The security of this protocol relies on the hardness of the discrete logarithm problem, which states that recovering the private exponent from the public key alone is computationally infeasible.

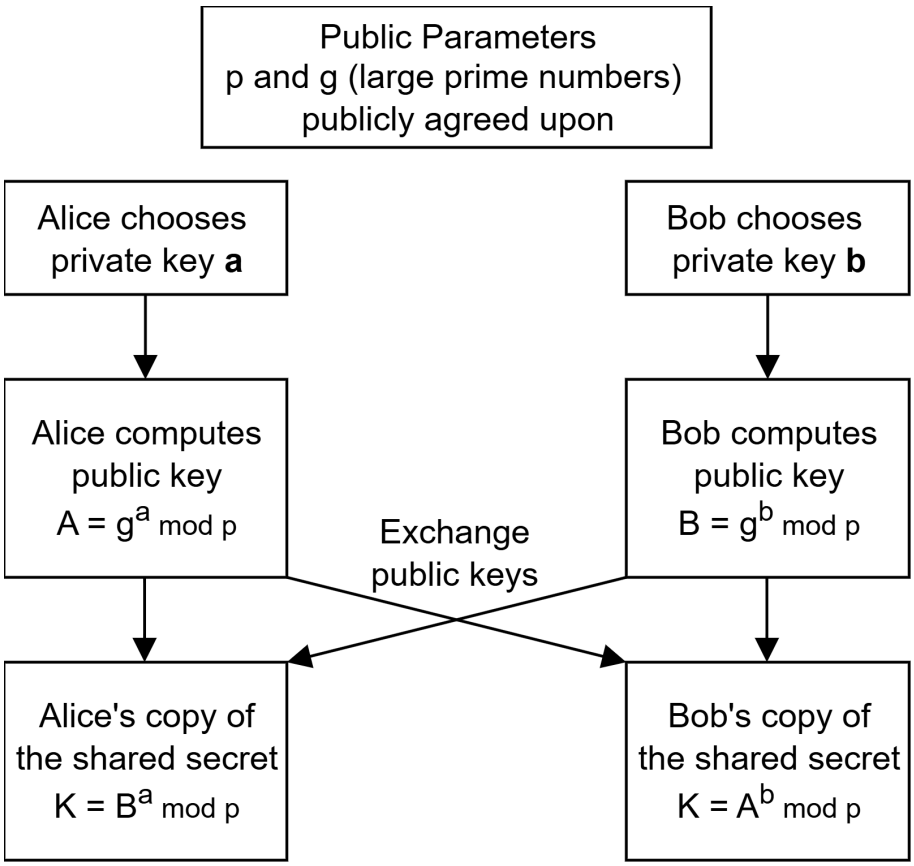

Figure 1: A simple view of key establishment using Diffie-Hellman.

These methods defined the first thirty years of digital security, but as computational power grew, the required key sizes for RSA and standard DH became unmanageable, leading to a need for more efficient mathematics.

2.1.2 The Shift to Elliptic-Curve Cryptography

In the early 2000s, many cryptographic protocols began transitioning toward Elliptic-Curve Cryptography (ECC). Instead of operating over finite fields of integers, ECC relies on the algebraic structure of elliptic curves over finite fields. The advantage of the Elliptic Curve Discrete Logarithm Problem (ECDLP) is that it is significantly harder to solve per bit of key size than the two previous mathematical problems that RSA and DH are based on.

For example, a 256-bit elliptic curve key, such as Curve25519 used by WireGuard, provides a security level equivalent to a 3072-bit RSA or DH key [15]. This efficiency is one of the primary reasons modern protocols have adopted ECC. Smaller key sizes reduce bandwidth requirements and computational overhead, enabling faster handshakes and smaller packet headers, which are particularly important for performance-sensitive systems such as VPN protocols.

Despite their efficiency, elliptic-curve systems remain fundamentally vulnerable to quantum attacks due to their reliance on discrete logarithm problems. This vulnerability forms one of the key motivations for the development of post-quantum cryptographic algorithms. Recognizing this risk, NIST has initiated a long-term transition toward quantum-resistant cryptographic algorithms. Current guidance recommends that organizations begin migrating away from classical public-key mechanisms in the coming years, with many traditional systems expected to be phased out after approximately 2030 as post-quantum standards are deployed [16]. Such a transition highlights the need to evaluate how existing protocols that rely heavily on elliptic-curve cryptography, such as WireGuard, can adapt to post-quantum cryptographic primitives.

2.1.3 Key Exchange vs. Key Encapsulation

It's important to distinguish between the two main methods for establishing shared symmetric cryptographic keys.

Traditional key exchange protocols such as Diffie-Hellman employ an interactive process in which both parties contribute to the generation of a shared secret. Instead of sending the secret directly, each party exchanges intermediate values derived from their private key, and both participants can independently compute the same shared key from the other's public value. This bidirectional structure is displayed in Figure 1.

In contrast, asymmetric post-quantum algorithms including ML-KEM and HQC are designed as Key Encapsulation Mechanisms (KEMs). Rather than a mutual exchange, KEMs follow a sender-receiver model. As shown in Figure 2, the sender generates a random symmetric key and encapsulates it using the recipient's public key, producing a ciphertext that is transmitted to the recipient. The recipient uses their corresponding private key to decapsulate the ciphertext and recover the shared secret. No interactive contribution from the recipient is required.

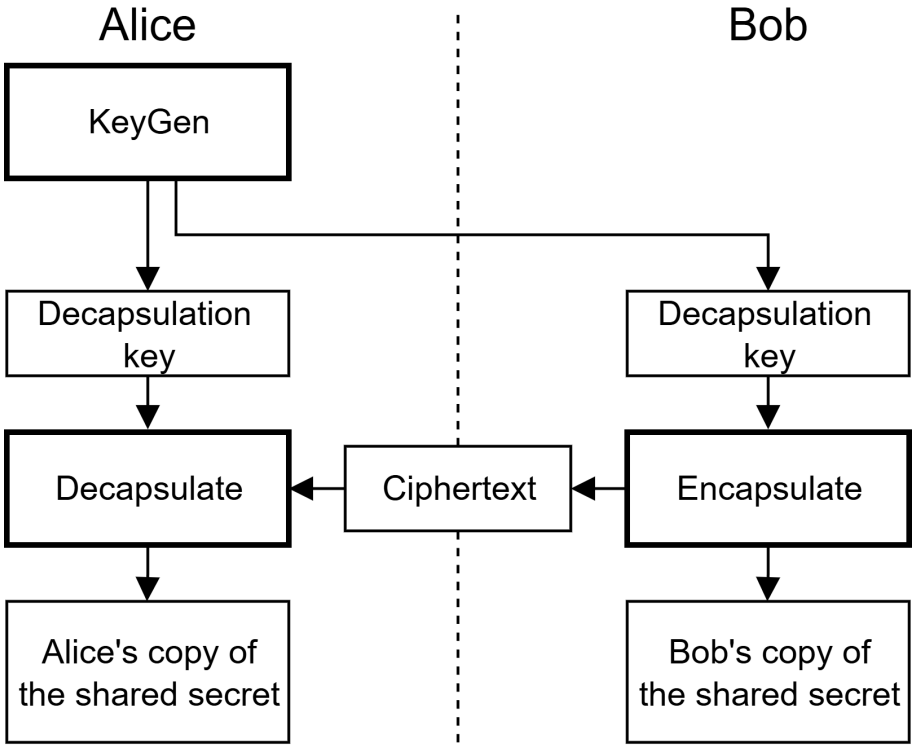

Figure 2: A simple view of key establishment using a KEM

While the end result is the same, both parties arrive at a shared secret, the logistical flow is different. This distinction is a core challenge when integrating post-quantum KEMs into existing protocols like WireGuard, which was originally designed for the interactive key exchange flow of Diffie-Hellman.

Despite this difference in their structure, KEMs are the flow used for post-quantum cryptography due to the nature of the mathematical problems on which post-quantum security is based. These do not support the bidirectional structure of a key exchange protocol. Instead, they lend themselves to a one-directional approach where one party can efficiently encapsulate a secret under a public key, while only the holder of the corresponding private key can recover it.

2.2 The Quantum Threat

As discussed in the introduction, the long-term security of modern cryptographic systems relies on the assumption that certain mathematical problems are computationally infeasible for classical computers to solve. However, the development of quantum computing challenges this assumption. Quantum computers operate according to the principles of quantum mechanics and are capable of executing algorithms that can outperform classical algorithms for specific types of problems.

There are two quantum algorithms that are especially relevant in this context: Shor's algorithm, which efficiently solves integer factorization and the discrete logarithm problem, and Grover's algorithm, which accelerates brute-force search. These algorithms are the frontrunners of the quantum threat to current cryptographic systems and are discussed in the following sections.

2.2.1 Quantum Computing

We must establish a high-level understanding of quantum computers work in order to understand how they are able to threaten current encryption.

A classical computer processes information using bits, binary values that exist in one of two states: 0 or 1. These bits are manipulated deterministically through electrical circuits made up of transistors and logic gates, which enables the computer to perform arithmetic and logical operations. At any given point in time, a classical system represents a single, well-defined state.

A quantum computer instead operates using quantum bits, commonly referred to as *qubits*, which has the ability to exist in a superposition of both states simultaneously. As a result, a qubit represents a linear combination of the two states rather than a single deterministic value. When multiple qubits are combined, the system can represent a probability distribution of many possible states at once. For a system of n qubits, this corresponds to 2^n possible configurations, resulting in an exponentially large state space.

The other key property of quantum systems is entanglement, where the states of multiple qubits become correlated in a way such that the state of one qubit cannot be described independently of the others. Entanglement enables quantum computers to capture and exploit relationships between variables in ways that are not possible in classical systems.

Quantum algorithms leverage superposition, entanglement, and the manipulation of probability amplitudes to perform certain classes of computations in a fundamentally different way than classical algorithms. By carefully controlling how probability amplitudes evolve during computation, these algorithms can amplify the likelihood of correct solutions while cancelling out incorrect ones. This interference structure is what enables quantum computers to achieve significant speedups over classical computation for problem classes where it can be meaningfully exploited [17].

2.2.2 Shor's Algorithm

One of the most significant breakthroughs in quantum computing for cryptography was the discovery of Shor's algorithm by Peter Shor in 1994 [2]. The algorithm provides an efficient quantum method for solving the two mathematical problems that are central to modern public-key cryptography: integer factorization and the discrete logarithm problem.

In order to fully grasp the impact of Shor's algorithm, let's first consider the best known classical algorithm for solving these problems. The General Number Field Sieve (GNFS) operates in sub-exponential but superpolynomial time [18], which means that every additional digit in an integer makes the factorization time disproportionately longer. While significantly faster than naive approaches, the computational cost still grows at an infeasible rate for larger key sizes. The largest RSA integer factored to date is RSA-250, an 829-bit number factored in February 2020 using CADO-NFS, an optimized software implementation of GNFS [19]. This required approximately 2,700 CPU core-years [20], illustrating that classical factoring remains computationally prohibitive at the key sizes used in practice.

Shor's algorithm completely demolishes this notion of infeasability. By leveraging the principles of quantum mechanics, it is able to factor large integers and solve discrete logarithms in polynomial time. It accomplishes this by reducing the problem into a period-finding problem. Given an integer N, the algorithm selects a random value and constructs a function whose period is related to the factors of N. While finding such periods is computationally difficult for classical computers, quantum computers can solve this efficiently using the Quantum Fourier Transform (QFT).

The practical implications of Shor's algorithm have been extensively explored and optimized in later research. A widely cited paper by Craig Gidney and Martin Ekerå demonstrated in 2021 that factoring a 2048-bit RSA integer could be feasible using a quantum computer with around 20 million physical qubits and 8 hours of computation time, assuming realistic error rates, cycle times and reaction times [21]. This paper has recently received a follow-up by Gidney alone where he demonstrated that the same task could accomplished in less than a week using less than a million noisy qubits under the same assumed constraints [22].

Thus, the existence of Shor's algorithm represents a fundamental threat to modern public-key cryptography. Since protocols such as TLS, SSH, and VPN systems rely on RSA, Diffie-Hellman, or elliptic-curve cryptography for secure key establishment, the availability of large-scale quantum computers would render these systems insecure.

2.2.3 Grover's Algorithm

Grover's algorithm [3] addresses a different class of problems than Shor's algorithm, specifically unstructured search. Given a search space of size N, a classical brute-force algorithm requires $\mathcal{O}(N)$ evaluations to find a target element. Grover's algorithm reduces this complexity to $\mathcal{O}(\sqrt{N})$ by leveraging quantum amplitude amplification.

In the context of cryptography, Grover's algorithm impacts symmetric-key primitives such as block ciphers and hash functions. But, unlike Shor's algorithm, Grover's algorithm does not completely break symmetric cryptographic schemes. Instead, it effectively halves their security level in terms of bit strength. This degradation can be mitigated by increasing key sizes. For example, a 128-bit symmetric key, which is considered secure against classical adversaries, would offer only 64 bits of security against a quantum adversary using Grover's algorithm. Consequently, doubling the key length (from 128 to 256 bits) is generally considered sufficient to maintain security in a post-quantum setting.

It is also important to note that practical implementations of Grover's algorithm face significant overhead due to error correction and circuit depth requirements, which further limits its near-term applicability. But it remains a critical consideration when evaluating the long-term security of symmetric cryptographic systems.

2.2.4 "Harvest now, decrypt later."

A common misconception about the quantum threat is that it only becomes relevant when sufficiently large quantum computers become available, but this view overlooks an already active risk model known as the "harvest now, decrypt later" (HNDL) [4].

In this model, an adversary intercepts and stores encrypted communication today

with the intention of decrypting them in the future once sufficiently powerful quantum computers exist. This strategy is particularly concerning for data with long-term confidentiality requirements, such as government communications, intellectual property, personal data, and critical infrastructure information.

It is important to understand what part of modern encrypted communication is vulnerable in this context. While data at rest is protected by symmetric encryption algorithms such as AES, which are considered resistant to quantum attacks, the asymmetric public-key algorithms used during the key exchange are the primary target. If an adversary can intercept the key exchange step of a session protected by TLS for example, they can store it and later use a quantum computer to recover the symmetric session key and decrypt all the session data. The implication is that even data which appears to be well-protected at rest may be vulnerable if the key exchange that established its protection was recorded.

The HNDL threat has driven significant regulatory action across multiple jurisdictions, each establishing strict timelines for the transition to post-quantum cryptography directly motivated by HNDL risk.

In the United States, the NSA's Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) establishes a phased migration schedule for National Security Systems. Traditional networking equipment such as VPNs and routers is required to support and prefer CNSA 2.0 algorithms by 2026, and to use them exclusively by 2030. Large-scale public-key infrastructure systems, such as TLS, follow a slightly later schedule, with mandatory preference by 2030 and exclusive use by 2033 [23].

In Europe, the European Commission published a coordinated PQC implementation roadmap, which establishes a timeline for the transition to PQC for all member states. Under this roadmap, all member states are expected to have begun transitioning to post-quantum cryptography by the end of 2026, with critical infrastructure systems fully transitioned to PQC no later than the end of 2030 [24].

At the standards level, NIST's deprecation timelines add further urgency. Classical quantum-vulnerable algorithms providing less than 128 bits of security are set to be deprecated after 2030 and disallowed after 2035, with algorithms above the 128-bit level set to follow the same eventual prohibition [25].

The "harvest now, decrypt later" threat establishes that the urgency of post-quantum migration is determined not only by the arrival of quantum computers, but also by the confidentiality requirements of data being transmitted and stored today. The combination of already-active data collection, long cryptographic migration timelines, and firm regulatory deadlines means that organizations handling sensitive data must treat the transition to quantum-resistant cryptography as an active and ongoing priority.

2.3 Post-Quantum Cryptography

Post-Quantum Cryptography (PQC) refers to cryptographic algorithms that are designed to remain secure against attacks from both classical and quantum computers. The goal is to replace or complement current cryptographic primitives such as RSA, Diffie-Hellman, and ECC. In many cases, PQC schemes are designed to be integrated

into existing protocols with minimal changes, although this still requires adaptations due to fundamental differences in protocol logic and accommodation for larger key sizes.

PQC algorithms are based on mathematical problems that are believed to be hard even for quantum computers. The main categories include lattice-based, code-based, multivariate, and hash-based cryptography. This thesis focuses on the two NIST-selected approaches: lattice-based cryptography (ML-KEM) and code-based cryptography (HQC).

2.3.1 Lattice-Based Cryptography (ML-KEM)

ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism), formerly known as CRYSTALS-Kyber [26], is a lattice-based KEM standardized by NIST in Federal Information Processing Standards (FIPS) publication 203 [6] in August 2024.

ML-KEM is based on the hardness of lattice problems, specifically the so-called Module Learning With Errors (MLWE) problem. The core idea is that a system of linear equations with small random errors is computationally hard to solve, even for a quantum computer.

One of the main advantages of ML-KEM is its strong performance and relatively moderate key sizes when compared to other PQC candidates. For example, ML-KEM-768 has a public key size of around 1184 bytes, which is manageable in protocols like TLS. Key generation and encapsulation are also significantly faster than RSA. These properties make ML-KEM suitable for practical deployment, which is why NIST selected it as the primary standard for post-quantum key establishment.

2.3.2 Code-Based Cryptography (Hamming Quasi-Cyclic)

Hamming Quasi-Cyclic (HQC) [7] is a code-based KEM selected by NIST in March 2025 as a backup standard to ML-KEM [27]. A draft standard is expected in 2026, with finalization planned for 2027.

The security of HQC is based on the hardness of decoding random quasi-cyclic codes, specifically the Quasi-Cyclic Syndrome Decoding (QCSD) problem. This problem is a structured variant of the general Syndrome Decoding (SD) problem, which is known to be NP-hard. Unlike older code-based schemes such as McEliece [28], HQC does not require hiding the structure of the code. Instead, it uses publicly known quasi-cyclic codes, and its security relies entirely on the hardness of the decoding problem itself.

HQC was selected primarily to provide *cryptographic diversity*. Since ML-KEM is based on lattice problems, a future cryptanalytic breakthrough targeting lattice-based cryptography could potentially affect all systems relying on ML-KEM. By standardizing HQC, which is based on a completely different mathematical foundation, NIST ensures that a backup remains available if ML-KEM is found to be vulnerable. NIST cited HQC's solid and well-analyzed construction as reasons it was preferred over other fourth-round candidates.

2.3.3 Hybridization Strategies

Because PQC algorithms are relatively new and have not yet undergone the same level of long-term cryptanalysis as classical schemes, a common deployment strategy is to combine a classical key exchange with a PQC KEM within a single handshake. This approach is known as a *hybrid key exchange*.

In a hybrid scheme, a classical Diffie-Hellman exchange and a PQC KEM are executed in parallel within the same handshake.

In the DH component, both parties contribute an ephemeral public key and independently compute the same shared secret. In the KEM component, the initiator encapsulates a secret to the responder's ephemeral public key, producing a ciphertext and a shared secret. The responder then decapsulates the ciphertext using their private key to derive the same shared secret. The two resulting secrets are then passed into a Key Derivation Function (KDF) to derive the final session key.

The key security property of this approach is that the resulting session is only compromised if *both* components are broken. If the PQC algorithm contains an unknown vulnerability, the classical component continues to provide security against current adversaries. Conversely, if a sufficiently powerful quantum computer becomes available in the future, the PQC component ensures that the key remains protected.

Hybrid key exchange is currently the industry-standard approach for deploying PQC in practice. It is already used in TLS 1.3, where hybrid mechanisms such as `X25519MLKEM768` (combining X25519 with ML-KEM-768) have been standardized by the IETF [29], and is being actively deployed by providers such as Cloudflare [30].

2.3.4 Forward Secrecy

Forward Secrecy (FS) is the property of a key establishment protocol where the compromise of long-term secrets used in the exchange does not expose the session keys of past communications.

In classical protocols, FS is achieved by generating fresh ephemeral DH keypairs for each session and discarding them immediately after use. Because the session key is derived from an ephemeral secret that is never stored, a future compromise of a session key cannot be used to reconstruct past session keys.

However, as discussed in Section 2.2.4, classical FS alone does not protect against a quantum-capable adversary. An adversary employing the HNDL strategy can record encrypted conversations today and later break the DH exchange retroactively using a sufficiently powerful quantum computer, rendering the ephemerality of the session keys irrelevant. It is therefore necessary that Post-Quantum Forward Secrecy (PQFS) in modern cryptographic contexts is achieved using quantum-resistant primitives.

In a hybrid key exchange, as described in the previous section, FS requires that both components contribute ephemeral keying material. Since the session key is derived from both components, one of which is a PQC KEM, PQFS holds as long as the KEM component remains secure, while the classical component continues to provide FS against non-quantum adversaries.

When PQC algorithms have withstood enough cryptanalytic scrutiny to justify

sole reliance on them, it may become viable to replace the hybrid construction with a purely PQC-based key exchange. PQFS would then be achieved entirely through ephemeral KEM keypairs, with no classical component required. The security guarantees would then hold against both classical and quantum adversaries without the overhead of maintaining two parallel key exchanges.

2.3.5 Practical Challenges

PQC algorithms introduce several practical challenges that must be addressed before they can be widely deployed.

The main challenge is the significantly larger message sizes compared to classical cryptography. PQC schemes typically produce larger public keys, ciphertexts, and signatures, which increases the size of protocol messages. This can create compatibility issues with existing systems, especially older implementations that impose strict limits on message sizes. These larger key materials may require adjustments to how handshake messages are structured and transmitted.

Another challenge is the integration of PQC algorithms into existing protocols, as these were originally designed for classical key-exchange primitives and do not always directly support the encapsulate/decapsulate model of PQC schemes. As a result, careful modifications are required to assure correct functionality and maintain security guarantees. In the case of TLS 1.3, for instance, changes are needed to accommodate larger key shares and to support hybrid key exchange mechanisms.

Finally, PQC implementations must be designed with strong resistance to side-channel attacks. Like classical cryptographic algorithms, PQC schemes can leak sensitive information through timing differences, power consumption, or electromagnetic emissions. Mitigating timing-based leakage in particular requires constant-time implementations, where the execution time of cryptographic operations does not vary with secret data. Besides timing, physical side-channels also present a practical threat. Recent research has demonstrated practical chosen-ciphertext side-channel attacks against HQC implementations [31], where an adversary with physical access submits carefully crafted ciphertexts and observes electromagnetic emissions during decapsulation to fully reconstruct the secret key. The attack targets the Reed-Muller decoding step of HQC's decapsulation, and the authors propose a masking-based countermeasure as a mitigation. These findings demonstrate why careful engineering practices are critical for any real-world PQC deployment.

3 WireGuard

WireGuard is a modern, free and open-source VPN protocol developed by Jason A. Donenfeld, and first publicly presented at the Network and Distributed System Security Symposium (NDSS) in 2017 [8]. It was conceived as a reaction to the complexity and bloat of existing VPN solutions such as IPsec and OpenVPN, that makes them notoriously difficult to configure correctly and whose large codebases make security auditing require significant effort.

WireGuard was designed with a different philosophy. Instead of supporting many possible cryptographic choices and configuration options, WireGuard uses a small and fixed set of modern cryptographic primitives. The protocol focuses on three main goals:

- Simplicity
- High performance
- Strong security

The primary C implementation of WireGuard is written in less than 4000 lines of code, excluding cryptographic primitives, compared to the 100 000 to 400 000 lines of the OpenVPN and IPsec implementations. This makes the codebase small enough for a single person to read and understand in full. which has direct benefits: a smaller attack surface, easier auditing, and eaiser to maintain.

Unlike protocols like IPsec or OpenVPN, WireGuard does not support cipher agility. Cipher agility means that the communicating peers can negotiate which cryptographic algorithms should be used during the connection setup. While this provides flexibility, it has historically introduced downgrade attacks and misconfiguration problems. WireGuard instead uses a fixed cryptographic suite consisting of:

- Curve25519 for key exchange
- ChaCha20-Poly1305 for authenticated encryption
- BLAKE2s for hashing
- HKDF for key derivation
- SipHash24 for internal hash table protection

WireGuard operates exclusively over UDP and uses a lightweight handshake mechanism to establish secure connections. A key design goal is to achieve fast connection establishment with minimal latency, normally requiring only one round-trip time (1-RTT).

Due to its strong performance and simplicity, WireGuard has become widely adopted. It has been integrated into the Linux kernel, is used as the underlying protocol in commercial VPN services such as NordVPN (NordLynx) and Mullvad, and is also widely deployed across mobile devices and cloud infrastructure.

3.1 The Noise Protocol Framework

WireGuard's handshake is built on the Noise Protocol Framework [32], a public-domain framework for constructing secure communication protocols based on Diffie-Hellman key agreement. Rather than defining a single fixed protocol, Noise provides a vocabulary of tokens that are arranged into message patterns. These message patterns are then arranged into handshake patterns that protocol designers can instantiate with concrete cryptographic functions.

A Noise handshake maintains several internal variables during the key establishment process. These variables store key values and current cryptographic state.

Each peer maintains the following variables:

- `s, e:` The local peer's static and ephemeral key pairs.
- `rs, re:` The remote peer's static and ephemeral key pairs, as learned during the handshake.
- `h:` A handshake hash that is updated with every piece of data sent or received. It accumulates a transcript of the entire handshake and is used as associated data in AEAD operations, ensuring that any modification to previously sent messages causes decryption to fail.
- `ck:` A chaining key that accumulates the output of every DH operation performed during the handshake via HKDF. Once the handshake completes, the final session keys are derived from `ck`, meaning they are cryptographically bound to every DH operation that took place.
- `k, n:` The current AEAD encryption key and nonce, derived from `ck` after each DH mix. These are used to encrypt and authenticate payloads embedded in handshake messages.

The Noise framework processes handshake messages token by token. Each token represents a cryptographic operation. The most important tokens used by WireGuard are:

- `"e":` The sender generates a fresh ephemeral key pair, transmits the ephemeral public key, and mixes it into `h`.
- `"s":` The sender encrypts its static public key under the current `k` with `h` as associated data and transmits the result. The ciphertext is then mixed into `h`.
- `"ee", "se", "es", "ss":` A DH operation is performed between the key pairs indicated by the two characters, where the first character refers to the local party's key and the second to the remote party's key (`e` for ephemeral, `s` for static). The resulting shared secret is mixed into `ck` via HKDF, and `k` and `n` are rederived.
- `"psk":` A pre-shared symmetric key is mixed into `ck`. This is an optional extension discussed further in Section 3.1.2

3.1.1 The Noise_IKpsk2 Handshake pattern

WireGuard specifically uses the Noise_IKpsk2 handshake pattern. The name encodes the structure of the exchange: `I` indicates that the initiator transmits its static public key during the handshake, `K` indicates that the responder's static public key is already known to the initiator before the handshake begins, and `psk2` indicates that a pre-shared key is mixed into the chaining key at the end of the second message. Because the responder's static key is known in advance, the initiator can perform a DH operation with it immediately, which allows the initiator's static key to be encrypted before transmission and provides identity hiding against passive observers.

```
IKpsk2:
<- s
...
-> e, es, s, ss
<- e, ee, se, psk
```

Figure 3: Abstract Noise_IKpsk2 handshake pattern diagram.

Reading the pattern line by line:

0. `<- s`: The responder's static public key `s` is provisioned out of band before the handshake begins.
1. `-> e, es, s, ss`: The initiator sends its ephemeral public key (`e`), performs a DH operation between its ephemeral key and the responder's static key (`es`), encrypts and sends its own static public key (`s`), and performs a DH operation between its static key and the responder's static key (`ss`).
2. `<- e, ee, se, psk`: The responder sends its ephemeral public key (`e`), performs a DH operation between the two ephemeral keys (`ee`), performs a DH operation between its ephemeral key and the initiator's static key (`se`), and finally mixes the pre-shared key into the chaining key (`psk`).

The resulting handshake achieves mutual authentication, forward secrecy through the ephemeral keys, and identity hiding for the initiator, all within a single round trip.

3.1.2 Pre-Shared Key

Noise supports an optional Pre-Shared symmetric Key (PSK) that when included, is mixed into the chaining key at a designated point in the handshake.

WireGuard exposes this parameter directly in its configuration. By default it is unused, but it can be set to inject an externally derived secret into the handshake. Out-of-band PQC solutions such as Rosenpass, discussed further in Section 4, perform a separate post-quantum key exchange and inject the resulting secret into WireGuard's PSK field, protecting session confidentiality even if the classical DH is later broken by a quantum computer.

3.2 Cryptographic Primitives

As mentioned in the beginning of this section, WireGuard is very opinionated about which cryptographic primitives it utilizes. This is a deliberate design choice as the ability to offer multiple algorithms at runtime, has historically been a source of downgrade attacks and misconfiguration vulnerabilities in protocols such as TLS and IPsec [33]. By removing the choice entirely, WireGuard eliminates this attack surface.

WireGuard currently uses the following set of modern cryptographic primitives:

- **Curve25519** for Diffie-Hellman key exchange. Curve25519 is an elliptic-curve DH function designed for both high performance and resistance to implementation errors. A static Curve25519 key is 32 bytes and provides approximately 128 bits of classical security.
- **ChaCha20-Poly1305** for authenticated symmetric encryption. ChaCha20 is a stream cipher and Poly1305 is a message authentication code, when combined they provide both confidentiality and integrity. This combination is particularly fast on hardware without AES acceleration, such as mobile processors.
- **BLAKE2s** for hashing. BLAKE2s is a fast cryptographic hash function used for key derivation and as part of the handshake's transcript hashing.
- **HKDF** for key derivation, built on top of BLAKE2s, used to derive the final session keys from the accumulated DH outputs.
- **SipHash-2-4** for internal hashtable keys. This protects WireGuard's internal data structures against hash-flooding denial-of-service attacks and is not part of the cryptographic handshake itself.

From a post-quantum perspective, ChaCha20-Poly1305, BLAKE2s, and HKDF are considered safe with their current parameters as symmetric primitives and hash functions are only weakened quadratically by Grover's algorithm. The vulnerable component is Curve25519, whose security depends on the elliptic-curve discrete logarithm problem, which Shor's algorithm solves efficiently. Consequently, the key exchange mechanism used by WireGuard is not considered secure against large-scale quantum computers and replacing or augmenting it is the central challenge of post-quantum WireGuard.

3.3 Handshake

WireGuard's handshake is a concrete instantiation of the Noise_IK pattern described in Section 3.1.1, augmented with WireGuard-specific fields for session management and denial-of-service resistance. The handshake consists of two messages: an initiation sent by the peer that wants to establish a session, and a response returned by the other peer. Once both messages have been exchanged, both sides independently derive a pair of symmetric session keys and data transfer can begin immediately, with no further round trips required. This is the 1-RTT property that makes WireGuard particularly efficient for short-lived or frequently rekeyed connections.

3.3.1 Initiation Message

The initiator begins by generating a fresh ephemeral Curve25519 key pair and initializing the handshake state. The chaining key and handshake hash are both set to a hash of the Noise protocol identifier string, which encodes the chosen handshake pattern and cryptographic primitives:

$$\begin{aligned} h_0 &= \texttt{Hash("Noise\_IKpsk2\_25519\_ChaChaPoly\_BLAKE2s")} \\ ck_0 &= h_0 \\ h_1 &= \texttt{Hash}(h_0 \| \texttt{"WireGuard v1 zx2c4 Jason@zx2c4.com"}) \end{aligned}$$

Finally, the responder's static public key S_R is mixed into h:

$$h_2 = \texttt{Hash}(h_1 \| S_R)$$

Both parties then begin the handshake with an identical state, and any deviation in the agreed protocol or peer identity is detectable from the very first message.

1. The ephemeral public key is appended to the message and mixed into `h`.
2. A DH is computed between the initiator's ephemeral private key and the responder's static public key. The result is mixed into `ck` via HKDF, and `k` is derived from the updated `ck`. This step binds the message to the responder's identity: only a party holding the corresponding private key can reproduce this value and decrypt what follows.
3. The initiator's static public key is encrypted under the current `k` with `h` as associated data and appended to the message. `h` is updated to include the ciphertext. This provides identity hiding: a passive observer cannot determine who the initiator is.
4. A DH is computed between the initiator's static private key and the responder's static public key. The result is again mixed into `ck`, providing static-key-based mutual authentication.
5. A timestamp is encrypted under the current `k` and appended. The responder checks that this timestamp is greater than any previously accepted timestamp from this initiator, which prevents replay of old initiation messages.
6. Two MAC fields, `mac1` and `mac2`, are appended for denial-of-service resistance. These are described in Section 3.3.4.

The initiation message structure is as follows:

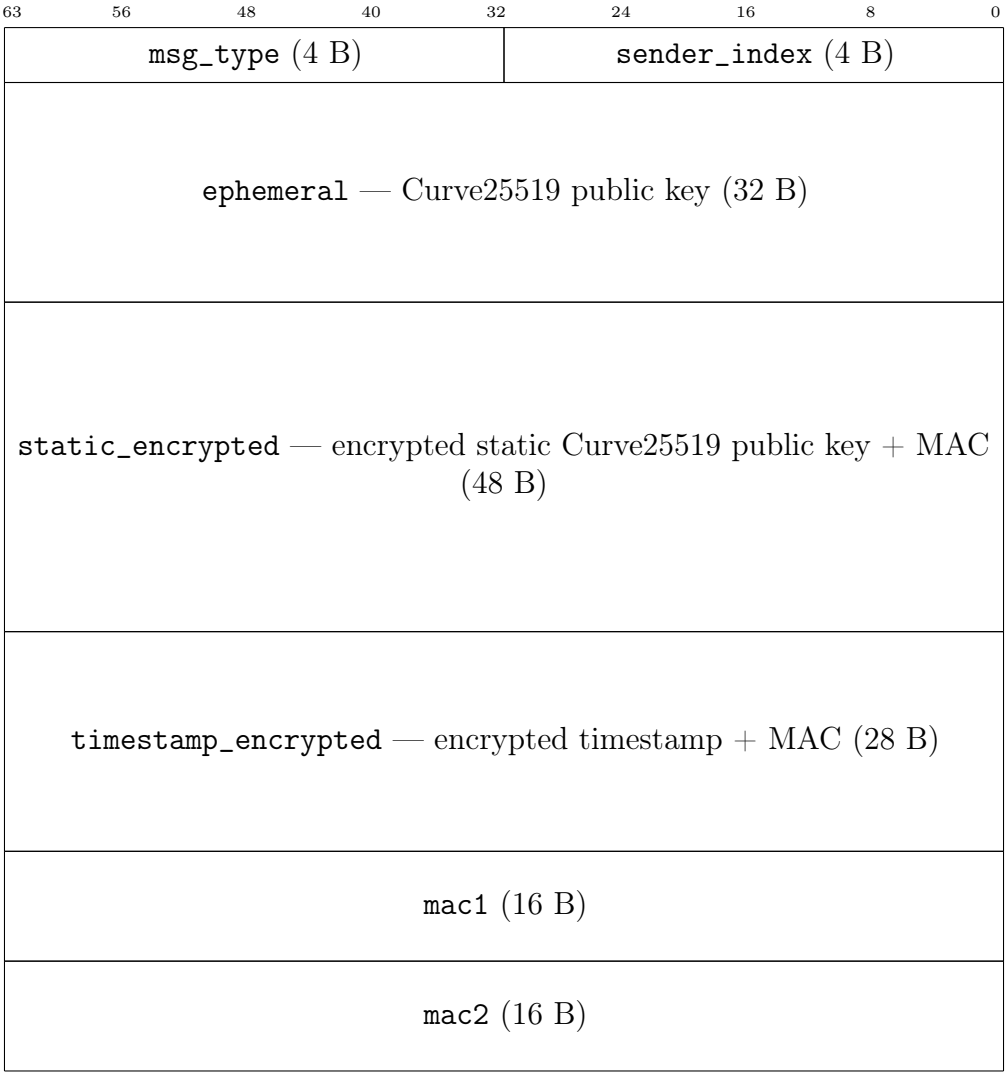

Figure 4: WireGuard handshake initiation message (148 bytes total)

3.3.2 Response Message

Upon receiving the initiation message, the responder reconstructs the same sequence of DH operations, verifying at each step that the decrypted material is consistent. If the `mac1` check fails, the message is dropped immediately. If any subsequent decryption step fails or the timestamp is not fresh, the message is discarded silently.

If the initiation message is valid, the responder generates its own fresh ephemeral key pair and constructs the response:

1. The responder's ephemeral public key is appended to the message and mixed into `h`.
2. A DH is computed between the two ephemeral keys and mixed into `ck`. Together with the initiator's ephemeral key from the first message, this establishes the ephemeral-ephemeral shared secret that provides forward secrecy.
3. A DH is computed between the responder's ephemeral key and the initiator's static key and mixed into `ck`. This authenticates the initiator's static identity to the responder.

4. The pre-shared key is mixed into `ck` at this point, contributing any externally injected secret to the final key material.
5. An empty encrypted payload is appended. Although it carries no plaintext, its AEAD tag covers the entire handshake transcript accumulated in `h`, authenticating the complete exchange.
6. `mac1` and `mac2` are appended as in the initiation.

The response message structure is as follows:

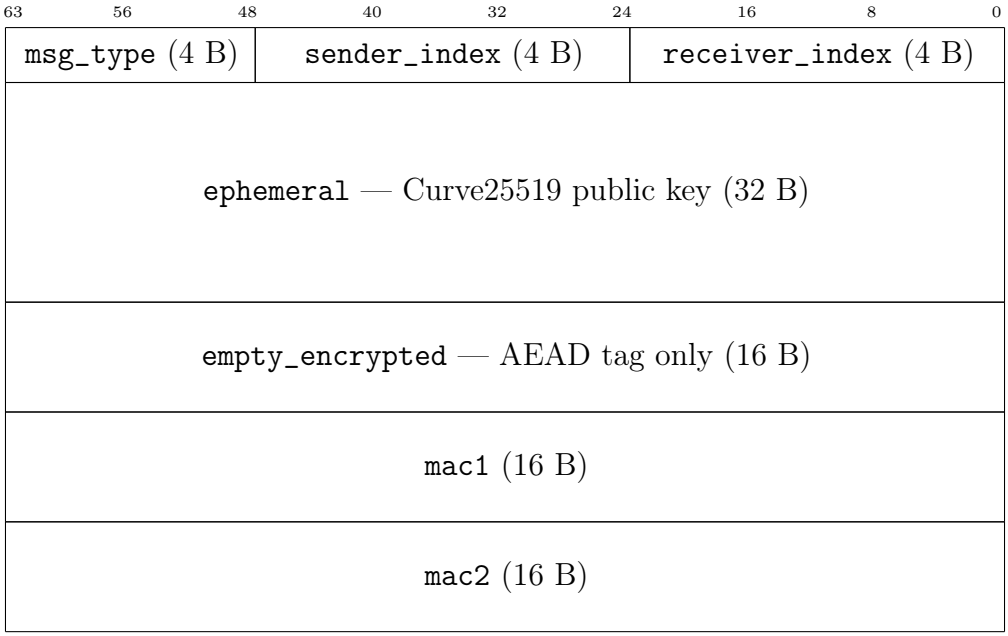

Figure 5: WireGuard handshake response message (92 bytes total)

3.3.3 Session Key Derivation

Once the initiator receives and validates the response message, both parties have performed the same set of DH operations and their chaining keys *ck* are identical. Two symmetric session keys are derived from *ck* using a final HKDF step: one key for packets flowing from initiator to responder, and one for the reverse direction. All subsequent data packets are encrypted with ChaCha20-Poly1305 using these keys. The handshake state is then discarded entirely to guarantee forward secrecy.

To maintain forward secrecy for long-lived connections, WireGuard rekeys automatically every 3 minutes of active use, or after 2^{64} packets have been sent during a session, whichever comes first. Each rekey involves a fresh handshake with new ephemeral keys, producing session keys that are independent of all previous sessions.

3.3.4 Cookie Mechanism

Every WireGuard handshake message carries two MAC fields, `mac1` and `mac2`, which together form WireGuard's defense against denial-of-service (DoS) attacks.

The `mac1` is always present and is computed as a keyed hash of the entire message content using a key derived from the recipient's static public key. Because computing

a valid `mac1` requires knowledge of the recipient's public key, it serves as a lightweight proof that the sender is aware of the intended recipient. Any packet with an invalid `mac1` is dropped immediately, before any DH computation takes place, making it cheap to filter out random or malformed packets under load.

`mac2` provides a second layer of protection against sustained floods. When the responder detects that it is under load, it enters a cookie-required state and sends a short-lived encrypted cookie to the initiator, derived from their source IP address using a secret that rotates every two minutes. Subsequent messages from that source must include a valid `mac2` computed from this cookie. Because the cookie is bound to the initiator's source IP and transmitted encrypted under their static public key, an attacker spoofing source addresses cannot benefit from cookies they did not receive, and an on-path observer cannot forge them.

Under normal conditions `mac2` is set to zero and ignored, making it a zero-cost mechanism in the absence of an active attack.

3.3.5 Security Proofs

The security of WireGuard's handshake has been formally analyzed in both the symbolic and computational models. Donenfeld and Milner provided a computer-verified proof in the symbolic model using the Tamarin prover [34], covering properties such as forward secrecy, identity hiding, and resistance to replay attacks. Symbolic proofs are useful for identifying logical flaws and can often be automated, but they model cryptographic primitives as idealized black boxes. As a result, they generally provide weaker guarantees than proofs in the computational model.

Dowling and Paterson later gave a computational proof [35], which provides stronger guarantees by making fewer idealizing assumptions about the underlying primitives. In doing so, they identified a subtle issue with the 1-RTT handshake. WireGuard uses the initiator's first transport message as implicit key confirmation, but because this packet is encrypted using the session keys themselves, formally proving both secrecy and confirmation becomes difficult. They proposed a solution that adds a small explicit confirmation message, encrypted under a separate key derived solely for that purpose. This change does not add an extra round trip either since the responder was already required to wait for a message from the initiator before sending any data of its own.

3.4 Stealth

A distinctive property of WireGuard is that it does not respond to any packet that fails cryptographic validation. There is no error message, no rejection notice, and no handshake failure notification. For a network scanner or an adversary probing the port, a WireGuard endpoint is indistinguishable from a host with no service running.

This property is a direct consequence of WireGuard's decision to operate exclusively over UDP with a fixed message format. A TCP-based VPN protocol such as OpenVPN and IPsec must at minimum complete a TCP handshake before any application-layer authentication can occur, exposing a connection-oriented attack surface. The only way to elicit any response from a WireGuard endpoint is to send

a cryptographically valid initiation message, which allows it to sidestep this problem entirely.

3.5 Key Management

WireGuard's identity model is deliberately simpler than that of TLS or IPsec. There are no certificates, no certificate authorities, and no public key infrastructure. Each WireGuard peer generates a static Curve25519 key pair, and manually configures their peers by adding each other's static public keys to their respective configuration files.

Each peer's configuration also specifies a set of *allowed IPs*: the IP address ranges that the peer is permitted to send and receive traffic for. This ties cryptographic identity directly to network routing. When a packet arrives, WireGuard looks up the source IP in the allowed IPs table to determine which peer's session key to use for decryption. If decryption succeeds, the packet is accepted. If it fails or no matching peer exists, the packet is dropped. The allowed IPs list essentially serves as both a routing table and an access control list.

An important implication of this model for post-quantum integration is that static public keys are provisioned out of band and do not need to be transmitted within the size-constrained UDP handshake messages. This means that even the large static public keys of certain PQC schemes such as Classic McEliece, which exceed 250 kB, could be used for static authentication without affecting handshake message sizes.

3.6 Go Implementation

While the main WireGuard implementation is a Linux kernel module written in C, a userspace implementation exists for platforms where kernel-level access is unavailable or undesirable. This is `wireguard-go` [36], the official userspace implementation written in Go, which exposes the same configuration interface as the kernel module and is used on macOS, Windows, iOS, and Android.

The main motivation for a userspace implementation in Go is portability and ease of integration. Go provides cross-platform support, a rich standard library, and easier interoperability with higher-level application code compared to kernel C code. This makes it straightforward to embed WireGuard as a library in a larger application, as is common in VPN clients.

However, userspace implementations come with a performance trade-off. Because each packet must cross the boundary between user space and the kernel's network stack through a TUN interface, there is additional overhead compared to the kernel module, where packet processing happens entirely in kernel space. For this reason, `wireguard-go` is recommended primarily for platforms where the kernel module is not available, and the kernel module remains the preferred choice on Linux servers.

A notable alternative to `wireguard-go` is BoringTun, a userspace implementation written in Rust that is developed by Cloudflare [37]. BoringTun is deployed on millions of iOS and Android consumer devices as well as thousands of Cloudflare Linux servers. Because Rust avoids garbage collection and provides fine-grained

control over memory, it can achieve better and more predictable performance than the Go implementation for packet-heavy workloads.

In the context of this thesis, `wireguard-go` is the implementation of choice because it allows PQC cryptographic libraries to be used directly with the WireGuard codebase without requiring kernel modifications. The message structs, DH calls, and key derivation steps are all clearly separated, making it possible to augment the handshake with additional KEM operations without restructuring the entire codebase. This makes `wireguard-go` significantly more accessible for experimental PQC integration than the C kernel module.

3.7 PQC Implementation Challenges

Integrating PQC into WireGuard introduces several challenges that stem from a fundamental architectural mismatch. WireGuard's handshake is built around the Diffie-Hellman key exchange, where both parties contribute a key share and independently derive the same shared secret. But since PQC schemes are typically structured as KEMs, where one party generates a shared secret and encrypts it for the other, this means that PQC algorithms cannot simply replace the DH steps in the existing handshake without overarching changes to its code.

A more immediate practical challenge is the size of PQC key material. A Curve25519 public key is 32 bytes, while an ML-KEM-768 public key is much larger at 1184 bytes and its ciphertext at 1088 bytes. This matters because WireGuard is specifically designed to keep its handshake messages small enough to fit within a single unfragmented UDP packet. For IPv4, the minimum guaranteed MTU along any path is 1500 bytes, and subtracting the IPv4 (20 bytes) and UDP (8 bytes) headers leaves only 1472 bytes for the payload.

Fitting a full PQC handshake within this budget is very difficult, and exceeding it forces fragmentation which causes the protocol to no longer maintain its 1-RTT handshake core design property.

4 Related Work

The foundational work on post-quantum WireGuard was presented by Hülsing, Ning, Schwabe, Weber, and Zimmermann at IEEE S&P 2021 [10]. Unlike most earlier work on post-quantum security for real-world protocols, their variant does not only address post-quantum confidentiality, but also targets full post-quantum authentication. To achieve this, they replace the Diffie-Hellman-based handshake with a more generic construction using only KEMs, resulting in a protocol that provides post-quantum security for both key exchange and identity authentication. Security is established in both the symbolic model, covering identity hiding and Denial-of Service (DoS) mitigation among other properties, and the computational model of Dowling and Paterson [35], in which they prove key indistinguishability.

The core design challenge they address is the MTU constraint described in Section 3.7: because WireGuard operates over UDP and assumes that handshake messages fit within a single unfragmented IPv6 packet, the key material contributed by PQC algorithms must fit within 1232 bytes. To satisfy this constraint, they instantiate their construction using Classic McEliece as the CCA-secure KEM and a passively secure variant of Saber, carefully selected so that the combined key material fits within the available space. A significant practical limitation of this choice is the large public key size of Classic McEliece, which substantially increases the server's memory footprint and complicates deployment in resource-constrained environments such as the Linux kernel.

PQ-WireGuard serves as an important performance reference point as the algorithms it uses, Classic McEliece and Saber, were the best available candidates at the time, but have since been superseded by the NIST-standardized ML-KEM and the newly selected HQC. This thesis revisits the performance question using these standardized algorithms, allowing for a more relevant comparison against the current state of post-quantum standardization.

Rosenpass [9], developed by Varner, Lipp, Schmidt, Dua, and Zaeske, takes a different approach to the integration problem. Rather than modifying the WireGuard handshake directly, Rosenpass runs as a separate process alongside WireGuard and performs a post-quantum key exchange independently. The resulting shared secret is then supplied to WireGuard as its pre-shared symmetric key (PSK), producing a hybrid connection where security holds as long as either the classical WireGuard handshake or the Rosenpass key exchange remains secure.

Such a sidecar architecture avoids any modification to WireGuard's codebase, making it practical to deploy today without kernel changes. The protocol builds on the PQ-WireGuard design but improves it by adding resistance against state disruption attacks, where an adversary replays the first handshake message to disrupt the responder's state. Rosenpass addresses this by returning an encrypted cookie to the initiator, so that the responder does not commit state until the initiator proves reachability. The implementation is written in Rust and uses Classic McEliece for long-term keys and CRYSTALS-Kyber-512 (Round 3 NIST) for ephemeral keys.

One important limitation of the PSK-injection strategy used by Rosenpass is that it only achieves post-quantum confidentiality as mentioned in Section 3.1.2. While Rosenpass itself provides post-quantum authentication via Classic McEliece, Wire-

Guard's own static authentication keys remain Curve25519-based. Thus, a quantum adversary running Shor's algorithm could still break authentication independently of Rosenpass. Achieving full post-quantum authentication would therefore require replacing the static key mechanism entirely, which the PSK slot is not designed to do.

For this thesis, Rosenpass is relevant as a deployed reference implementation that uses CRYSTALS-Kyber in a production context. Its PSK-based integration strategy also represents a natural performance baseline. Since it adds a separate handshake on top of WireGuard, the combined latency is higher than a fully integrated solution. The performance results in this thesis can therefore be interpreted in the context of whether a deeper integration actually yields a meaningful latency improvement over the Rosenpass approach.

The most recent related work is by Hashimoto, Katsumata, Niot, and Wiggers, to appear at IEEE S&P 2026 [11]. This paper adresses the original PQ-WireGuard design by Hülsing et al. [10], and improves it across three dimensions: protocol design, computational security, and efficiency.

On the design side, the paper introduces the novel idea of *reinforced KEMs* (RKEMs), which provide a formal way to replace DH operations within the Noise framework with KEM operations. The original PQ-WireGuard handled this substitution in an ad-hoc manner, and the RKEM abstraction formalizes and corrects the way KEM outputs are absorbed into the handshake transcript and key derivation chain.

On the security side, the paper extends the computational security analysis beyond what was covered in the original work by Hülsing et al., which only proved key indistinguishability in the computational model and relied on the symbolic model for the remaining properties. The new work formalizes these additional security notions such as entity authentication, DoS security, and identity hiding, directly in the computational model. It also fixes a subtle issue in the prior computational model of both Hülsing et al. and Dowling and Paterson [35], arising from the mid-protocol handling of pre-shared keys in the post-specified peer model. They do this by explicitly splitting the responder algorithm into an identification phase and a key derivation phase, so that the pre-shared key is only assigned once the initiator's identity has been established.

On the efficiency side, the RKEM-based redesign eliminates the need for Classic McEliece entirely. By using two instances of ML-KEM, the large public key sizes and associated memory overhead of the original design are avoided. Crucially, the design exploits WireGuard's out-of-band static key provisioning, described in Section 3.5, that large static public keys does not have to be transmitted within the handshake messages. This makes the protocol significantly more practical for deployment.

The paper represents the current state of the art for formally analyzed post-quantum WireGuard designs, and it uses ML-KEM as its primary building block, the same algorithm that is evaluated in this thesis. While this thesis does not implement the RKEM construction, the performance characteristics of ML-KEM measured here are directly relevant to understanding the practical cost of the approach proposed by Hashimoto et al.

5 State of the art

The NIST Post-Quantum Cryptography Standardization Process [5] continues in parallel with this thesis. ML-KEM was finalized as FIPS 203 in August 2024 and is the algorithm evaluated in this work. HQC was selected as a backup standard in March 2025 and is currently undergoing draft standardization, with a final standard expected in 2027 [27]. The ongoing standardization of HQC is relevant as the performance results for HQC reported in this thesis reflect the current reference implementation, not a final and optimized standard.

Post-quantum key exchange is actively being deployed in the Transport Layer Security (TLS) protocol, which represents the most widely monitored early indicator of PQC adoption at scale. The IETF has published a draft standard for hybrid key exchange in TLS 1.3 using `X25519MLKEM768`, which combines X25519 and ML-KEM-768 into a single key share [29].

Cloudflare began deploying `X25519MLKEM768` in TLS connections in early 2024 and published an analysis of its performance and adoption trajectory [30]. Their findings indicate that hybrid ML-KEM adds minimal latency overhead for typical HTTPS connections, largely because TLS key material is transmitted in-band and the computational cost of ML-KEM is low. This supports the premise of this thesis that ML-KEM is viable in latency-sensitive protocols, though WireGuard imposes stricter constraints than TLS due to its MTU-bound message model. The TLS deployment data also provides an external reference against which the handshake latency results of this thesis can be contextualized.

6 Method

6.1 Research Approach

This thesis addresses the problem statement and research questions stated in Section 1.1 through a combination of qualitative and quantitative research. Figure 6 illustrates the overall research process from the initial problem statement to the final evaluation.

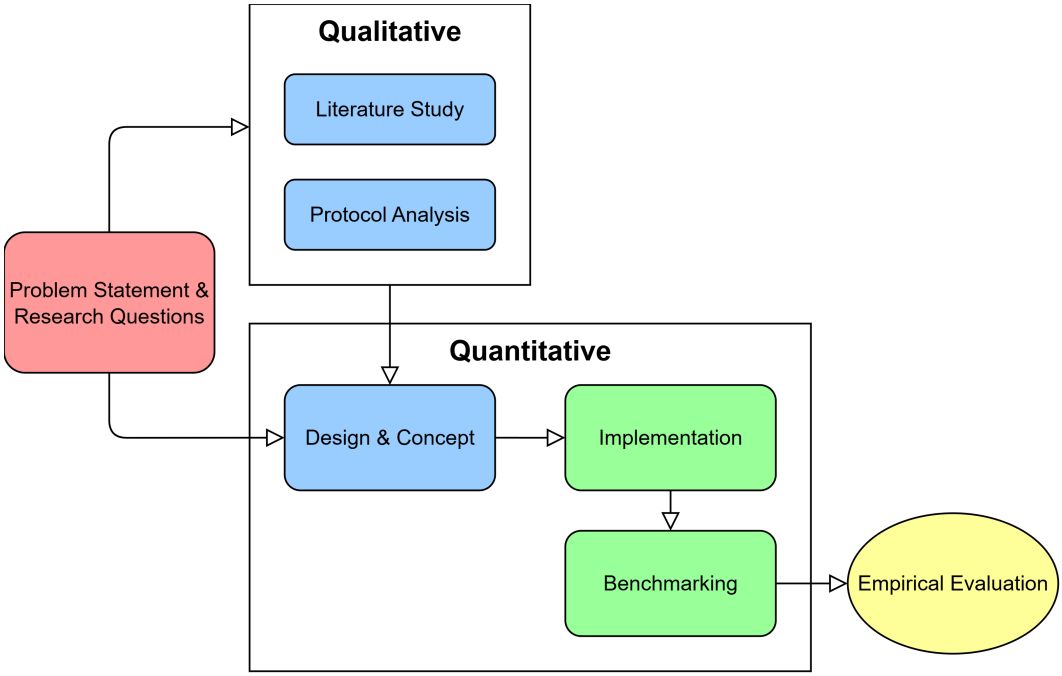

Figure 6: Overview of the research process.

As shown in Figure 6, the process begins with the problem statement and research questions, which drives both phases of the work. The qualitative phase begins with a literature study covering post-quantum cryptography standardization, the WireGuard protocol, and prior work on post-quantum WireGuard. A protocol analysis, in which WireGuard's internals are examined in depth, is then done in order to gain a solid understanding of the PQC integration challenges described in Section 3.7.

The quantitative phase builds on both the problem statement and the qualitative findings. It begins with a design and concept stage in which the literature study and protocol analysis are translated into concrete decisions about which algorithms to evaluate, which handshake modes to implement, and how to integrate KEM operations into the protocol. This is followed by the implementation in wireguard-go, then a controlled benchmarking test, and finally an empirical evaluation in which the results are analyzed and interpreted in the context of the research questions and related work.

6.2 Literature Study

Primary sources were consulted across the three areas mentioned above in Section 6.1: post-quantum cryptography standardization, the WireGuard protocol, and prior

post-quantum WireGuard work. For post-quantum cryptography, these were the NIST standardization documentation [5], [27], the ML-KEM and HQC algorithm specifications [6], [7], and background literature on lattice-based and code-based cryptography. For WireGuard, the primary source was the original paper by Donenfeld [8], supplemented by the Noise Protocol Framework specification [32]. The three prior post-quantum WireGuard papers described in Section 4 were the primary references for design decisions [9], [10], [11].

Sources were selected based on direct relevance: only primary research papers, official standards documents, and whitepapers from the implementing organizations were used as the basis for design decisions. Secondary sources such as blog posts and news articles were used only for contextual framing, not for technical claims.

The literature study served two purposes: it informed the selection of algorithms and handshake modes to evaluate, and it provided the performance reference points against which the empirical results in Section 7.5 are compared.

6.3 Protocol Analysis

The protocol analysis phase examined WireGuard's code base to identify exactly where and how post-quantum KEM operations could be integrated. The primary source for this analysis was the wireguard-go source code [36].

The analysis focused on three questions:

- Which parts of the handshake are tied to Diffie-Hellman and must to be modified?
- What constraints does the protocol impose on the size and structure of handshake messages?
- What changes to the existing codebase are necessary to support variable-length KEM key material without breaking the existing classical handshake path?

Four DH operations were identified in the classical handshake: `ephemeral-static`, `static-static`, `ephemeral-ephemeral`, and `static-ephemeral`. These are the operations that provide key agreement and mutual authentication, and they are the targets for KEM-based replacement or augmentation. The `static-static` DH operation is particularly significant because it has no direct KEM equivalent in an interactive setting, which is the reason why a pure-KEM handshake requires a more comprehensive redesign.

For message size and structure, the analysis established that WireGuard's fixed-size message fields are sized exactly for 32-byte Curve25519 keys, meaning that PQC key material would require a structural change over a simple substitution. With a standard 1500-byte Ethernet MTU and IPv4/UDP headers, the maximum payload budget per handshake message is 1472 bytes, as described in Section 3.7. Each candidate algorithm's key and ciphertext sizes were then compared against this budget to determine which variants would fit within a single unfragmented packet.

Examining the codebase revealed that the fixed-size array fields used for the classical 32-byte Curve25519 keys would need to be replaced with variable-length slices to accommodate PQC key material, and that the UAPI protocol would need new fields

for provisioning KEM public keys out of band. It also confirmed that the handshake code in `noise-protocol.go` was modular enough to add KEM branches without restructuring the entire file.

The insights gained from this phase was an understanding of the integration requirements that directly shaped the design decisions described in Section 6.4.

6.4 Design Choices and Rationale

6.4.1 In-Handshake Integration Rather Than PSK Injection

Rather than following the PSK injection approach taken by Rosenpass, the implementation focuses on integrating PQC directly into the WireGuard handshake. PSK injection, while practical and easy to deploy, only achieves post-quantum confidentiality and does not address authentication. Since the static Curve25519 keys used for peer authentication remain in place, a quantum adversary can still break authentication even if PSK injection is used. A deeper in-handshake integration is necessary to evaluate the full scope of the post-quantum transition, including both confidentiality and authentication.

The trade-off is that in-handshake integration requires non-trivial modifications to WireGuard's internals, which makes the implementation more invasive and harder to ship in production. This is an acceptable cost in the context of a research implementation intended to produce empirical measurements.

6.4.2 Both Hybrid and Pure-KEM Modes

Two distinct handshake variants were implemented: a hybrid mode that runs an additional KEM operation alongside the existing Curve25519 exchange, and a pure-KEM mode that replaces the Curve25519 exchange entirely. The two modes address different threat models and different points in the post-quantum migration timeline.

The hybrid mode targets the current deployment environment, where PQC algorithms are new and have not yet received the same degree of cryptanalytic scrutiny as classical schemes. Running both algorithms in parallel ensures Forward Secrecy as mentioned in Section 2.3.4, that the session is compromised only if both components fail simultaneously, thus providing a conservative security guarantee during the transition period. The hybrid mode does not achieve post-quantum authentication, since the static keys remain classical.

The pure-KEM mode uses KEM-based operations for both key agreement and identity authentication, achieving full post-quantum security at the cost of larger messages and more significant protocol restructuring. It corresponds more closely to the long-term vision of a fully post-quantum WireGuard and allows the additional overhead of full PQ authentication to be measured directly.

6.4.3 Userspace Implementation

The implementation was built using the `wireguard-go` userspace implementation. This choice was primarily motivated by practicality as integrating a C-language foreign-function interface to liboqs into the kernel module would require substantially

more development time and is not necessary for a research evaluation. The userspace implementation provides the same handshake logic and is representative of the protocol behavior, but its performance numbers differ from the kernel module due to the additional TUN interface overhead inherent to userspace packet processing.

6.4.4 Algorithm Selection

All six non-classical variants evaluated in this thesis use NIST-selected security levels, specifically ML-KEM-768 and ML-KEM-1024 (NIST security levels 3 and 5 respectively), and HQC-192 and HQC-256 (levels 3 and 5). These levels correspond to 192-bit and 256-bit post-quantum security, which match the NSA Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) requirements for national security systems [23]. ML-KEM-512 and HQC-128, which target 128-bit security, are implemented in the codebase but not benchmarked as they fall below the CNSA 2.0 threshold.

6.4.5 Modularity Over Performance

The implementation is designed to be modular and extensible rather than optimized for performance. A central `pqc.Config` struct encodes the active mode and the chosen KEM instance, and all handshake code branches on this configuration rather than being hardcoded for a specific algorithm. This means that new KEM algorithms can be added simply by implementing the `kems.KEM` interface and registering a new entry in the mode registry, without modifying any handshake logic. The cost of this design is dispatch overhead that a production-optimized version would not have.

6.5 Implementation

To support both the hybrid and pure-KEM handshake variants, the `wireguard-go` codebase was extended with a `pqc` package and a `pqc/kems` subpackage. The `kems` package defines a `KEM` interface with methods for key generation, encapsulation, and decapsulation, along with accessor methods for key and ciphertext sizes. Concrete KEM types (`mlkem512`, `mlkem768`, `mlkem1024`, `hqc128`, `hqc192`, `hqc256`) embed a shared `liboqsKEM` base type that calls into the liboqs library, and each type provides only the constant size values and the algorithm name that differ between variants. This design means that the liboqs foreign-function interface code is written once and shared across all algorithms.

The `pqc` package exposes a `Config` type that is constructed from a mode name string (for example, `"hybrid-mlkem768"` or `"mlkem1024"`) and stored on the `Device` struct. The active `Config` is read by the handshake code to determine which mode and algorithm to use. The mode name is also read from the `WIREGUARD_KEM` environment variable at startup, making it easy to run the same binary in different modes without recompilation.

6.5.1 Liboqs

The implementation uses liboqs-go [38], the Go binding for the Open Quantum Safe (OQS) project's liboqs C library [39], to provide the ML-KEM and HQC implementations. liboqs is a widely used open-source library for prototyping and

experimenting with post-quantum cryptography. Its algorithm implementations are derived from the reference and optimized code submitted by algorithm teams to the NIST PQC standardization project. Using the liboqs library ensures that the benchmark results reflect the performance of well-vetted, architecture-optimized algorithm implementations closely aligned with the NIST submissions.

One consequence of using a C library through cgo is that memory returned by liboqs resides in C-managed memory that becomes invalid after the library's cleanup function `handle.Clean()` is called. The implementation therefore copies all liboqs return values into Go-managed slices before the cleanup deferred call runs, avoiding use-after-free errors that would otherwise be silent in production.

6.5.2 Hybrid Mode

As described in Section 3.1.1, the classical handshake performs four DH operations spread across the initiation and response messages. The hybrid variant keeps all of these intact and adds a single ephemeral KEM exchange on top, without modifying any of the existing Curve25519 steps. Figure 7 illustrates the full hybrid handshake flow.

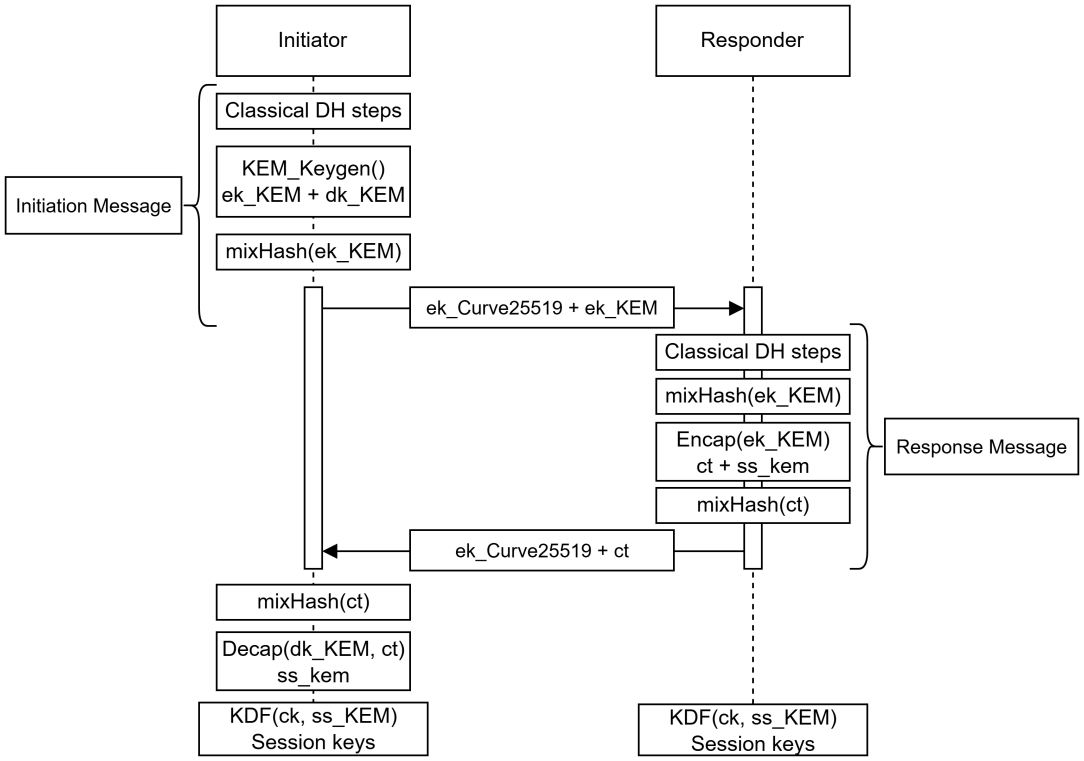

Figure 7: A flow diagram of a hybrid handshake, showing both the classical DH path and the KEM path running in parallel.

In the initiation message, after the classical transcript steps have been completed, the initiator generates a fresh ephemeral KEM keypair. The ephemeral KEM public key is appended to the `Ephemeral` field of the initiation message immediately after the 32-byte Curve25519 public key, and mixed into the handshake transcript hash. The private key is stored in the handshake state for later use.

In the response message, after the classical response steps have been completed, the responder reads the initiator's ephemeral KEM public key from the extended `Ephemeral` field, mixes it into the transcript hash, encapsulates a fresh shared secret to it, and appends the resulting ciphertext to the `Ephemeral` field of the response message. The ciphertext is mixed into the transcript hash, and the shared secret is stored temporarily in the handshake state.

Upon receiving the response, the initiator reads the KEM ciphertext from the response's `Ephemeral` field, mixes the ciphertext into its local copy of the transcript hash, and decapsulates it using the stored ephemeral private key.

At session key derivation time, the KEM shared secret is not mixed directly into the chain key during the handshake. Instead, it is combined with the classical chain key through a final `KDF` step immediately before the session keys are derived. This means that the session keys depend on both the classical DH outputs and the KEM shared secret, and the session is compromised only if an adversary can break both components. The hybrid handshake therefore guarantees post-quantum Forward Secrecy. If the KEM is broken, classical security remains, and if the classical DH is broken by a quantum adversary, the KEM compensates.

6.5.3 Pure-KEM Mode

The pure-KEM handshake is intended as a complete replacement of the Noise_IKpsk2 handshake. As shown in Figure 8, the exchange follows the same two-message structure but replaces every DH operation with a KEM operation. A different construction string (`Noise_KEM_IKpsk2_ChaChaPoly_BLAKE2s`) is used, and the algorithm name is additionally mixed into the initial transcript hash so that sessions using different algorithms cannot be confused.

In the initiation message, the initiator first mixes the responder's static public key into the transcript hash as identity commitment. Then generates a fresh ephemeral KEM keypair, mixes the ephemeral KEM public key into the hash and chain key, and includes it in the message. The initiator then encapsulates a shared secret to the responder's static KEM public key, producing a ciphertext (`ct_es`) that first mixed into the transcript hash and then also included in the message. This shared secret is mixed into the chain key, which means the subsequent encryption of the initiator's own static KEM public key can only be decrypted by the legitimate responder. Lastly, the initiator's static KEM public key is encrypted against the transcript hash and included in the message, which allows the responder to look up their peer entry.

On receiving the initiation, the responder mirrors the initiator by mixing their own static KEM public key and the received ciphertext `ct_es` and ephemeral KEM public key into their copy of the transcript hash, and also mixing the ephemeral key into their chain key. The responder then decapsulates `ct_es` using its static private key, reconstructs and mixes the shared secret into the chain key, and decrypts the initiator's static KEM public key to look up the peer. The responder encapsulates a second shared secret to the initiator's ephemeral KEM public key, producing `ct_ee`, which provides forward secrecy. The ciphertext is mixed into the transcript and the shared secret into the chain key. The responder also mixes its own static KEM public key into the transcript after the encrypted empty payload, creating an identity

commitment that the initiator can verify without the static key being transmitted again.

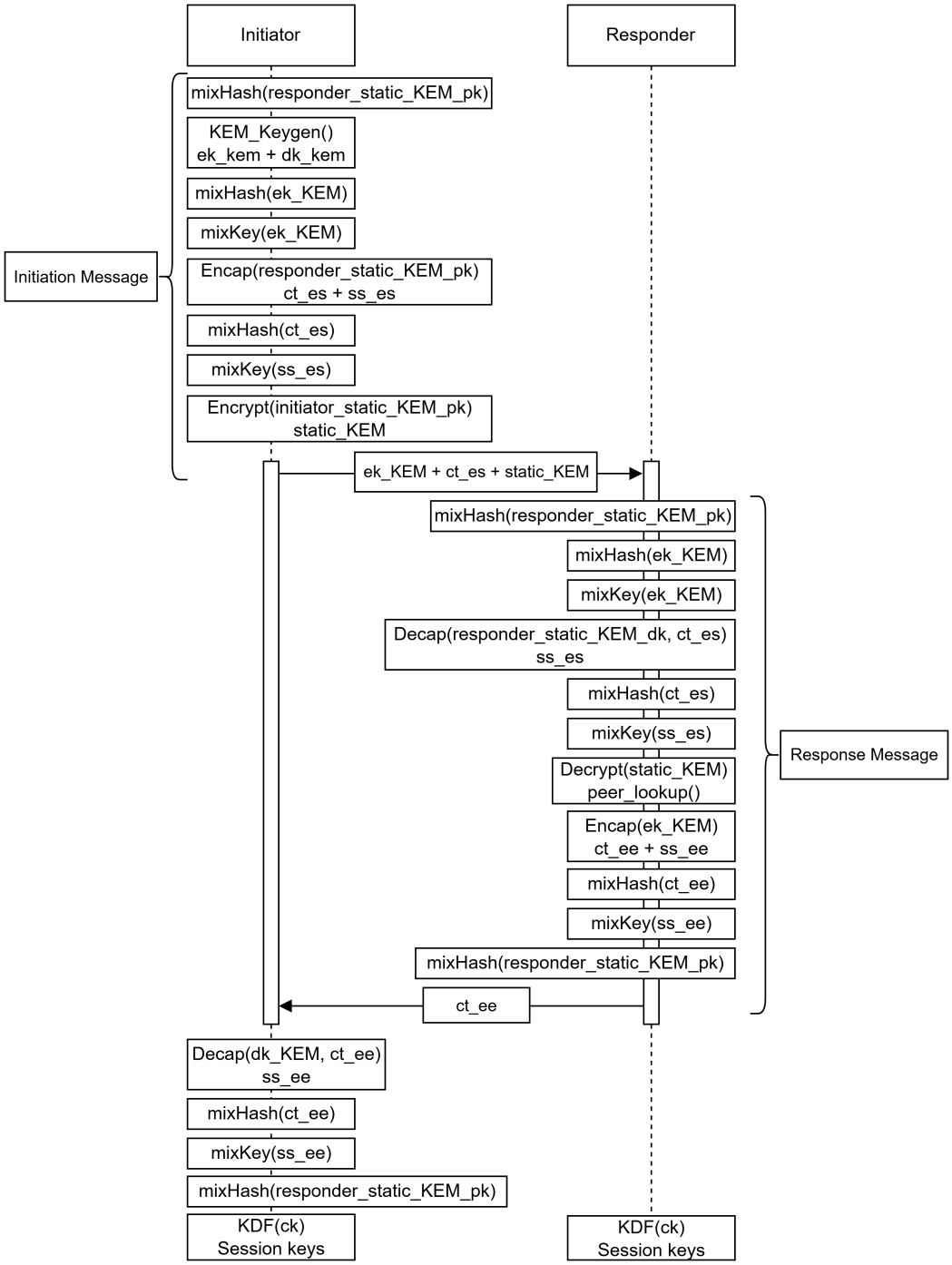

Figure 8: Flow diagram of the pure-KEM handshake. All DH operations from the classical handshake are replaced by KEM encapsulation and decapsulation operations.

On receiving the response, the initiator decapsulates `ct_ee`, mixes the ciphertext into the transcript and the shared secret into the chain key, verifies the AEAD on the empty payload, and mirrors the identity commitment by mixing the expected peer KEM public key into its local transcript. If both sides agree on the responder's identity, their chain keys will be identical and session key derivation succeeds.

This design achieves full post-quantum authentication because both the key agreement and the identity commitments are based solely on KEM operations over post-quantum public keys.

6.5.4 Message Structures and Serialization

Because the sizes of KEM public keys and ciphertexts vary by algorithm and are not known at compile time, the message structures for both variants uses variable-length slice fields instead of fixed-size arrays. The sizes of these fields are precomputed at device initialization time from the active `Config` and stored as fields on the `Device` struct. The serialization and deserialization code uses these stored sizes to correctly delimit the fields when reading from and writing to the wire.

The `Config` type also exposes helper methods: `MessageInitiationSize()`, `MessageResponseSize()`, `InitiationEphemeralSize()`, etc. that compute the expected wire size of each message for the active mode. These are used to pre-allocate correctly sized buffers and to validate incoming message lengths before attempting to parse them.

6.5.5 UAPI Extensions

WireGuard's userspace API (UAPI) was extended to support the provisioning, retrieval and expiration of KEM public keys for both the local device and each configured peer. A `kem_public_key` field was added to both the device-level and peer-level sections of the UAPI protocol. On input, the field accepts a Base64-encoded public key and validates its length against the active KEM's expected public key size. On output, the field is emitted alongside the standard Curve25519 public key when a KEM public key has been configured. A secondary lookup map, `kemKeyMap`, was added to the `peers` struct to support peer lookup by KEM public key, which is used in the pure-KEM handshake to identify the initiator from its encrypted static KEM public key.

6.6 Measurement Methodology

Handshake latency was measured using a ping-based RTT method. After forcing a new handshake by expiring the session keys via WireGuard's UAPI interface, a single `ping` was issued from the initiator to the responder. Because the session keys has been invalidated, the ping cannot be delivered until a full handshake completes, meaning that the measured round-trip time captures the complete handshake latency plus the ICMP echo time. This approach requires no log parsing, no cross-VM clock synchronization, and no modification to WireGuard's internal logging.

Baseline RTT

To isolate the handshake time from the underlying network overhead, a rolling baseline RTT was maintained throughout the benchmark. Every 100 trials, while the session is still live and no key expiry has been issued, five consecutive pings are sent and their mean is recorded as the current baseline. This per-trial baseline was then subtracted from each handshake RTT to produce the net handshake time reported in the results. Using a rolling baseline rather than a single measurement

taken at startup accounts for any drift in network conditions over the course of a long benchmark run.

Warmup

Before the recorded trials began, 100 warmup handshakes were performed and discarded. This ensured that any initial overhead from process startup, memory allocation, or cache warming did not influence the results. A fresh baseline sample was taken after the warmup phase to reflect the system's steady-state network conditions.

Recorded Trials

Each benchmark run consisted of 5000 recorded handshake trials per algorithm variant. Each trial followed the same sequence:

1. Expire the session keys
2. Issue a single ping to initiate a handshake
3. Record the RTT

A trial was considered failed if the ping did not return within 15 seconds, and any run in which more than 5% of trials fail was aborted. From the recorded values, mean, standard deviation, median, and 95th and 99th percentile latencies were computed. The handshake time reported in Table 1 is the mean of the baseline-subtracted RTTs, with the standard deviation reported in parentheses.

6.7 Benchmarking Setup

All benchmarks were performed using two identical virtual machines running Debian 12, hosted on the same KVM hypervisor. Each VM was configured with four virtual CPU cores, each mapped to a single-threaded socket, based on a QEMU Virtual CPU (x86_64, family 15, model 107) running at 2,4 GHz, with 3,8 GiB of RAM allocated. Notably, AES-NI hardware acceleration was available within the virtualized environment and used during the benchmarks [40].

The VMs were connected via a virtual switch, with each network interface configured with a standard 1500-byte MTU. One VM acted as the WireGuard initiator and the other as the responder for the duration of all benchmarks. Since both VMs reside on the same physical host, the network path between them is entirely virtual, eliminating real-network variables such as jitter and packet loss. This provides a stable and reproducible environment for isolating cryptographic overhead, though it means the results do not capture the additional latency that would be present in a real-world deployment across a physical network.

Furthermore, since traffic between the two VMs is handled entirely by the hypervisor's virtual switch, UDP fragmentation and reassembly occurs in software without the packet loss that fragmented datagrams can experience on a real network path. This means that handshake variants whose messages exceed the 1500-byte MTU will complete successfully in this environment despite violating WireGuard's design assumption of unfragmented delivery, and their measured latency does not reflect the reliability penalty that fragmentation would incur in practice.

6.8 Reliability, Validity, and Generalizability

6.8.1 Qualitative Research

The literature study and protocol analysis were conducted using primary sources only, with the selection rationale described in Section 6.2. In qualitative research, reliability refers to the consistency of the analytical procedures and is dependent on maintaining a clear decision trail so that another researcher could arrive at similar conclusions [41]. Since all sources used are publicly available and the reasoning connecting them to the design decisions in Section 6.4 is made explicit, the qualitative phase is considered reliable in the sense described by Runeson and Höst [42].

Validity in qualitative research concerns the precision with which the findings accurately reflect the underlying material [41]. The protocol analysis was conducted directly against the wireguard-go source code rather than a secondhand description of it. A risk to validity in any literature study is selective reading, this was partially mitigated by engaging with all three prior post-quantum WireGuard papers regardless of their approach, including Rosenpass whose PSK injection strategy was not followed, with the reasoning made explicit in Section 6.4.

Generalizability in qualitative research, or applicability in Lincoln and Guba's terminology, applies to whether findings can be transferred to other contexts or settings [41]. The findings here are specific to wireguard-go and the current WireGuard handshake design, so direct generalization is limited. However, the analytical approach and the design rationale documented in Section 6.4 transfer to other Noise-based protocols and future algorithm integrations.

6.8.2 Quantitative Research

Reliability is concerned with to what extent the data and the analysis are dependent on the specific researchers [42]. The benchmark is fully automated and the setup is documented in Section 6.6 in sufficient detail to allow reproduction on equivalent hardware. The 5000-trial sample size and reported standard deviations also allow the reader to assess the stability of the results.

Internal validity relates to the extent to which the design and analysis may have been compromised by confounding variables [43]. The measured RTT includes overhead beyond the handshake itself, notably the underlying network latency between the two VMs. The rolling baseline subtraction addresses this by isolating the cryptographic contribution from that overhead latency. Since both VMs are on the same physical host, network conditions are stable and uniform across all variants, reducing the likelihood that observed latency differences are caused by network variation rather than cryptographic cost.

External validity relates to the extent to which any conclusions can be generalized [43]. The results in this thesis are specific to x86_64 virtual hardware, and performance on other architectures may differ due to varying instruction set support. Additionally, the virtual network path means that over-MTU handshake variants complete without fragmentation penalties that would occur on a real network, so the latency figures for those variants should be treated as lower bounds rather than realistic deployment estimates.

7 Results

This section presents the empirical results from the benchmark as described in Section 6.6. The results cover handshake latency, message sizes, and memory footprint for each of the nine evaluated variants: classical WireGuard, four ML-KEM variants (hybrid and pure, at NIST security levels 3 and 5), and four HQC variants (hybrid and pure, at NIST security levels 3 and 5). The results are then used to answer the research questions posed in Section 1.1 and to draw comparisons with the related work.

7.1 Benchmark Metrics

Variant	Initial [Bytes]	Response [Bytes]	Baseline RTT [ms]	Handshake Time [ms]
Classic	148	92	0.522 (0.041)	1.820 (0.245)
Pure-MLKEM-768	3540	1148	0.511 (0.034)	1.172 (0.231)
Pure-MLKEM-1024	4788	1628	0.521 (0.052)	1.437 (0.243)
Hybrid-MLKEM-768	1332	1180	0.525 (0.063)	2.228 (0.283)
Hybrid-MLKEM-1024	1716	1660	0.517 (0.035)	2.374 (0.336)
Pure-HQC-192	18106	9038	0.529 (0.047)	364.038 (14.302)
Pure-HQC-256	28995	14481	0.520 (0.044)	668.182 (23.451)
Hybrid-HQC-192	4670	9070	0.549 (0.059)	201.687 (8.220)
Hybrid-HQC-256	7393	14513	0.538 (0.062)	365.557 (14.777)

Table 1: Handshake benchmark results. Values in parentheses are standard deviations. Handshake time is the mean baseline-subtracted RTT over 5000 trials.

Table 1 outlines the following values for each variant:

- **Initial [Bytes]:** The on-wire size of the initiation message in bytes.
- **Response [Bytes]:** The on-wire size of the response message in bytes.
- **Baseline RTT [ms]:** The mean RTT of five consecutive pings issued on an already-established session, representing pure network overhead with no handshake. The standard deviation of the baseline is shown in parentheses.

- **Handshake Time [ms]:** The mean baseline-subtracted RTT over 5000 trials, reflecting the duration of the handshake itself. The standard deviation is shown in parentheses.

The baseline RTT is consistent at approximately 0,5 ms across all nine variants, which confirms that the underlying virtual network conditions did not change between runs and that differences in handshake time reflect cryptographic overhead rather than measurement noise.

7.2 Handshake Latency

Variant	Mean [ms]	p50 [ms]	p95 [ms]	p99 [ms]
Classic	1.820	1.778	2.131	2.575
Pure-MLKEM-768	1.172	1.115	1.630	1.909
Pure-MLKEM-1024	1.437	1.383	1.893	2.233
Hybrid-MLKEM-768	2.228	2.148	2.822	3.187
Hybrid-MLKEM-1024	2.374	2.282	3.068	3.496
Pure-HQC-192	364.038	365.444	385.547	400.494
Pure-HQC-256	668.182	670.529	702.505	723.505
Hybrid-HQC-192	201.687	202.425	215.335	226.457
Hybrid-HQC-256	365.557	366.525	388.525	402.361

Table 2: Mean, 50th percentile (p50), 95th percentile (p95), and 99th percentile (p99) baseline-subtracted handshake latency for the classical, ML-KEM and HQC variants.

7.2.1 ML-KEM

A surprising result in the latency data shown in Table 2 was that both pure ML-KEM variants had a faster handshake time than classical WireGuard across all measurements. Pure-MLKEM-768 is approximately 36% faster, and Pure-MLKEM-1024 about 21% faster. This result seems counterintuitive at first as a handshake with larger messages should not be faster than the optimized classical baseline, but the explanation is found in the benchmark setup. As noted in Section 6.7, AES-NI hardware acceleration was available in the virtual machines and used during the benchmark. ML-KEM uses AES-256 in counter mode as an extendable output function during key generation, which maps directly onto AES-NI instructions. The classical X25519 key exchange is based on elliptic-curve arithmetic that receives no benefit from AES-NI, instead relying on general-purpose integer multiply instructions. On this hardware, ML-KEM's encapsulation and decapsulation are therefore faster than the X25519's scalar multiplication, which is why removing the DH operations with KEM operations reduces the total computation time despite the larger message sizes.

On hardware without SIMD support, such as embedded processors or older CPUs, the result would likely be quite different. The performance advantage of pure ML-KEM

over the classical handshake is therefore specific to this instance of hardware and should not be taken as a general expectation.

The hybrid ML-KEM variants, however, are slower than both the classical handshake and the pure-KEM variants. Hybrid-MLKEM-768 has a mean of 2,228 ms and Hybrid-MLKEM-1024 has a mean of 2,374 ms, representing handshakes approximately 22% and 30% slower the classical baseline respectively. This is expected since the hybrid mode keeps all the existing DH operations from the classical handshake and adds a full KEM encapsulation and decapsulation on top.

7.2.2 HQC

The HQC results are in a completely different performance category. Pure-HQC-192 has a mean handshake time of 364 ms and Pure-HQC-256 even reaches as high as 668 ms. The hybrid variants are faster but still far outside any acceptable range, Hybrid-HQC-192 takes 202 ms and Hybrid-HQC-256 takes 366 ms.

The main cause of this latency is not HQC's computational cost but the IP fragmentation and software reassembly forced by its large message sizes. HQC's public keys and ciphertexts are large enough that every HQC handshake message must be split into multiple IP fragments before transmission. As described in Section 6.7, the virtual network path in this benchmark handles fragmentation and reassembly entirely in software, without hardware offloading. The operating system must buffer all incoming fragments, wait for the final fragment to arrive, and then reassemble the complete datagram before WireGuard can begin processing the message. This reassembly overhead grows with the number of fragments and is the primary driver of the observed latencies, which is consistent with the fragment counts discussed in Section 7.3.3. The standard deviations shown in Table 1 for the HQC variants further support this notion. Pure-HQC-192 has a high standard deviation of 14,3 ms and Pure-HQC-256 has 23,5 ms, compared to the low standard deviation of less than 0,35 ms for all ML-KEM variants.

7.3 Message Size

Table 1 shows a large variance in the message sizes across the nine variants. To understand where these sizes come from, it helps to look at the individual fields that make up each message.

7.3.1 Initiation Message Fields Breakdown

A fixed overhead of 84 bytes is identical across all nine variants, as the fields it covers are independent of the choice of algorithm. As illustrated in Figure 4, these fields are the 4-byte message type, the 4-byte sender index, the 28-byte encrypted timestamp with its authentication tag, and the two 16-byte MAC fields used for denial-of-service resistance.

The three remaining fields differ considerably between the modes. In the classical handshake the initiator sends only its 32-byte Curve25519 ephemeral public key and a 32-byte encrypted version of its static Curve25519 key, for a total of 148 bytes. For the hybrid variants, the ephemeral key field is extended by appending the KEM

Variant	Ephemeral key	Ciphertext	Static	Fixed
Classic	32	0	32	84
Pure-MLKEM-768	1184	1088	1184	84
Pure-MLKEM-1024	1568	1568	1568	84
Hybrid-MLKEM-768	1216	0	32	84
Hybrid-MLKEM-1024	1600	0	32	84
Pure-HQC-192	4522	8978	4522	84
Pure-HQC-256	7245	14421	7245	84
Hybrid-HQC-192	4554	0	32	84
Hybrid-HQC-256	7277	0	32	84

Table 3: Breakdown of initiation message fields into ephemeral fields and fixed protocol overhead.

ephemeral public key directly after the classic Curve25519 key, and the static field remains a 32-byte Curve25519 key because authentication still relies on the classical identity.

The significantly larger field sizes of the pure-KEM variants arises from the need for three KEM components to be present in the message simultaneously. As mentioned in Section 6.5.3, the initiator sends its ephemeral KEM public key so the responder can encapsulate the ephemeral shared secret, a ciphertext `ct_es` produced by encapsulating to the responder's static KEM public key, and an encrypted copy of its own static KEM public key so the responder can identify the peer.

7.3.2 Response Message Fields Breakdown

Variant	Ephemeral key	Ciphertext	Static	Fixed
Classic	32	0	0	60
Pure-MLKEM-768	0	1088	0	60
Pure-MLKEM-1024	0	1568	0	60
Hybrid-MLKEM-768	32	1088	0	60
Hybrid-MLKEM-1024	32	1568	0	60
Pure-HQC-192	0	8978	0	60
Pure-HQC-256	0	14421	0	60
Hybrid-HQC-192	32	8978	0	60
Hybrid-HQC-256	32	14421	0	60

Table 4: Breakdown of response message fields into ephemeral fields and fixed protocol overhead.

The response message is simpler than the initiation in all modes. As shown in Figure 5, the message consists of a fixed header and a variable KEM section. The fixed portion accounts for 60 bytes: the 4-byte message type, 4-byte sender index, 4-byte receiver index, 16-byte empty encrypted payload with its authentication tag, and the two 16-byte MAC fields.

In the hybrid modes, described in Section 6.5.2, the responder keeps its Curve25519 ephemeral key and additionally sends the KEM ciphertext `ct_ee`, which encapsulates a shared secret to the initiator's ephemeral KEM public key.

Because the classical DH exchange has been removed entirely, the pure-KEM response contains no Curve25519 ephemeral key. Instead, the responder encapsulates a fresh shared secret to the initiator's ephemeral KEM public key, producing the ciphertext `ct_ee`, and returns this alongside the fixed overhead fields.

Response messages are consistently smaller than the initiation messages in the pure-KEM modes, by a factor of around two to three depending on the algorithm. This asymmetry arises as the initiator bears the cost of sending its ephemeral public key, the authentication ciphertext `ct_es`, and its encrypted static public key, while the responder only needs to return the single ciphertext `ct_ee`. In the hybrid modes this asymmetry is noticeably smaller, since both messages carry only a Curve25519 ephemeral key alongside a single KEM component.

7.3.3 Message Fragmentation

Variant	Init fragments	Resp fragments	Total fragments
Classic	1	1	2
Pure-MLKEM-768	3	1	4
Pure-MLKEM-1024	4	2	6
Hybrid-MLKEM-768	1	1	2
Hybrid-MLKEM-1024	2	2	4
Pure-HQC-192	13	7	20
Pure-HQC-256	20	10	30
Hybrid-HQC-192	4	7	11
Hybrid-HQC-256	6	10	16

Table 5: Number of IPv4 fragments required for each message at WireGuard's 1472-byte constraint.

Because WireGuard runs over UDP, any message exceeding the 1472 byte IPv4 payload limit defined in Section 3.7 must be split into multiple IP fragments and reassembled at the receiver before WireGuard can process it. Table 5 shows how many fragments each variant requires per message direction.

Among the benchmarked post-quantum variants, Hybrid-MLKEM-768 is the only one that fits within a single packet in both directions, matching the classical handshake in total packet count. The remaining ML-KEM variants all require a small number of additional fragments, with the response total consistently lower than the initiation due to the asymmetry discussed in the previous section.

The HQC variants tell a very different story: both the initiation and response messages require multiple fragments. Even the hybrid HQC variants are heavily fragmented in the response direction because the large HQC ciphertexts must be transmitted. The handshake latency results in Table 2 are a direct consequence of

this fragmentation, and the gap between the ML-KEM and HQC variants in Table 5 mirrors the gap observed in latency.

7.4 Memory and Storage

7.4.1 Per-Peer Static Key Storage

Each peer entry stores a 32-byte Curve25519 static public key. This stays the same for the hybrid variants, as the hybrid mode does not introduce a static KEM key and authentication still relies on Curve25519. The pure-KEM mode replaces this with a static KEM public key. These static keys are provisioned out of band before any handshake can take place, and must remain resident in memory for the server to perform peer lookup during incoming handshakes.

Variant	Key size	100 peers	10k peers	100k peers
Classic / Hybrid	32 B	3,1 kB	313 kB	3,1 MB
Pure-MLKEM-768	1184 B	115 kB	11,3 MB	113 MB
Pure-MLKEM-1024	1568 B	153 kB	15,0 MB	153 MB
Pure-HQC-192	4522 B	442 kB	43,2 MB	432 MB
Pure-HQC-256	7245 B	708 kB	69,1 MB	691 MB

Table 6: Per-peer static public key storage requirements and estimated totals at three deployment scales.

Table 6 shows that at small peer counts the choice of algorithm has no meaningful impact on storage. At 10,000 peers the differences become visible but the pure ML-KEM variants remain well within the capacity of a typical server. The pure HQC variants grow considerably more, with HQC-256 requiring around 220 times more storage than the classical case at this scale. At 100,000 peers the gap widens further, and the pure HQC variants represent a storage overhead that could be a concern on memory-constrained hardware.

7.5 Comparisons With Related Work

7.5.1 PQ-WireGuard

Hülsing et al. report a client-side handshake time of around 1,0 ms over a virtual 10 Gbps link with no added latency [10], which is in the same order of magnitude as the pure ML-KEM variants in Table 2. PQ-WireGuard fits both handshake messages within the 1472 byte IPv4 budget, which the pure ML-KEM variants in this thesis do not achieve, as shown in Table 5. The Classic McEliece parameter set used produces a static public key of 512 KB, which, at 10,000 peers amounts to approximately 5 GB for the peer table, compared to roughly 11 MB for pure ML-KEM-768 at the same scale (Table 6).

7.5.2 Rosenpass

A direct latency comparison with Table 2 is not meaningful for Rosenpass [9], since it operates as a separate sidecar process whose cryptographic cost must be added

to WireGuard's handshake time. Rosenpass fits both its InitHello and RespHello messages within a single IPv6 packet. Out of the variants in this thesis, only Hybrid-MLKEM-768 achieves a comparable result, and only under IPv4 (Table 5).

7.5.3 Rebar

Rebar [11] was benchmarked with 30,9 ms of added artificial latency and reports a handshake time of approximately 36,9 ms for the initiator, compared to 33,6 ms for classical WireGuard in the same environment. Its initiation and response messages are 1052 bytes and 1088 bytes respectively, fitting within the 1232 byte IPv6 budget that none of the pure-KEM variants in this thesis achieve.

7.6 Research Questions

RQ1: How do NIST-selected post-quantum key encapsulation mechanisms, specifically ML-KEM and HQC, affect the performance of a WireGuard handshake?

As shown in Table 2, ML-KEM improves handshake performance in the pure-KEM mode on the benchmark hardware, driven by AES-NI optimized implementations that outperform X25519 on this type of CPU. The hybrid ML-KEM mode adds modest overhead over the classical baseline. HQC produces latencies orders of magnitude above the classical baseline across all four variants, primarily due to IP fragmentation and software reassembly rather than cryptographic computation.

RQ2: What impact do ML-KEM and HQC have on resource usage in WireGuard, particularly in terms of message size and storage requirements?

ML-KEM increases message sizes significantly in the pure-KEM mode, as shown in Table 1, but the hybrid ML-KEM-768 variant fits within a single IPv4 packet in both directions. Per-peer static key storage for the pure ML-KEM variants, shown in Table 6, grows substantially compared to the classical case but remains manageable at realistic deployment scales. The HQC variants produce significantly larger messages and requires a far greater storage capacity at all parameters.

RQ3: To what extent can ML-KEM and HQC be integrated into WireGuard without violating its core design principles, such as low latency and small handshake size?

Hybrid-MLKEM-768 is the only post-quantum variant that preserves WireGuard's single-packet handshake under IPv4. The pure ML-KEM variants require a small number of fragments but remains computationally fast. HQC cannot be integrated without significantly violating WireGuard's design principles: the messages require heavy fragmentation and the resulting latencies shown in Table 2 are impractical.

RQ4: How do hybrid cryptographic approaches compare to purely post-quantum approaches in terms of security, performance, and deployability?

The hybrid approach preserves classical security as a fallback and provides post-quantum forward secrecy, but does not achieve full post-quantum authentication. The pure-KEM approach provides both. As shown in Table 2, the pure ML-KEM

mode is faster than the hybrid mode on this hardware because it replaces rather than augments the classical operations, though this result is hardware-dependent. The hybrid mode is more deployable right now because it does not require distributing new static KEM public keys for all peers and can be integrated without replacing the existing static key infrastructure.

RQ5: How does a hybrid WireGuard implementation using ML-KEM and HQC compare to existing post-quantum solutions in terms of performance and practical deployment?

Compared to Rosenpass, the in-handshake integration in this thesis completes post-quantum key establishment within a single WireGuard handshake rather than requiring a separate process, and the pure-KEM variants achieve full post-quantum authentication that Rosenpass does not provide. The cost is a higher deployment barrier, since the integration requires modifying WireGuard's codebase, and larger initiation messages compared to Rosenpass as shown in Table 1. Compared to PQ-WireGuard and Rebar, the pure ML-KEM variants produce larger messages that exceed the IPv6 budget, but offer a significantly lower per-peer storage requirement as shown in Table 6.

RQ6: What trade-offs exist between security level and ciphertext size when selecting PQC algorithms for use in WireGuard?

For ML-KEM, moving from NIST security level 3 to level 5 increases message sizes and per-peer storage moderately, as shown in Tables 1 and 6, and adds a small amount of latency visible in Table 2. The handshake remains practical at both levels. For HQC, the same step produces a much larger increase in both message size and latency, making an already impractical result considerably worse. For WireGuard specifically, where message size is a hard constraint, the level 3 to level 5 upgrade is manageable with ML-KEM but effectively prohibitive with HQC.

8 Discussion

This section outlines the contributions of the thesis, examines the practical consequences of the findings, and highlights the key constraints imposed by WireGuard's design. It further compares the implementation against existing post-quantum WireGuard solutions, reflects on the ethical and sustainability implications of the transition to post-quantum cryptography, and aims to direct future research. As this field is constantly evolving, some observations reflect the state of the art at the time of writing, and the landscape may change as both PQC implementations and the NIST standardization process mature.

8.1 Contributions

8.1.1 Contribution to Literature

This thesis contributes to the literature on post-quantum protocol integration by providing a empirical comparison of two NIST-selected KEMs, ML-KEM and HQC, within the same WireGuard codebase and under the same measurement conditions. Prior work evaluated different algorithms in isolation or relied on separate implementations, making direct performance comparisons difficult. The results here demonstrate concretely that algorithm choice has a decisive impact on WireGuard compatibility. The thesis also extends the literature by evaluating both hybrid and pure-KEM handshake modes at two NIST security levels, producing a more complete picture of the security-performance trade-off space than previous studies. Finally, the analysis of the pure-KEM mode's full post-quantum authentication property and its associated deployment costs adds to the understanding of what a complete quantum-resistant WireGuard deployment would require.

8.1.2 Contribution to Practice

This thesis contributes to practice by providing a working open-source implementation of post-quantum WireGuard in `wireguard-go` that supports both hybrid and pure-KEM modes for ML-KEM and HQC. The implementation can serve as a reference point for evaluating PQC migration options for VPN infrastructure. The benchmark results provides concrete latency and message-size figures that can be used to assess the operational impact of each variant before committing to a migration path. The finding that Hybrid-MLKEM-768 fits within the standard IPv4 MTU and adds only around 22% to handshake time provides a clear choice for operators who need to address the "harvest now, decrypt later" threat today. They can do so with this variant without replacing their existing static key infrastructure or accepting significant performance degradation.

8.1.3 Managerial Contribution

From a managerial perspective, this thesis supports decision-making for organizations that must plan their cryptographic migration under regulatory timelines such as CNSA 2.0 and the European Commission's 2030 deadline. The results show that a hybrid ML-KEM deployment is achievable within current WireGuard infrastructure without a full system redesign, which lowers the barrier for an initial migration. The comparison of per-peer storage requirements across deployment scales gives capacity

planners a direct way to estimate the infrastructure impact of switching from classical to post-quantum static keys. The clear performance gap between ML-KEM and HQC also simplifies algorithm selection for organizations running WireGuard under standard MTU constraints.

8.2 MTU Problem

The 1472-byte IPv4 payload budget is the single most constraining property of WireGuard for post-quantum integration. All prior work on post-quantum WireGuard has treated it as a hard constraint, and the algorithm selection in PQ-WireGuard [10] and Rebar [11] was shaped primarily by the requirement to stay within this budget.

As the results in Table 5 show, only one post-quantum variant in this thesis, Hybrid-MLKEM-768, fits within a single IPv4 packet in both message direction, making it the only variant that fully preserves WireGuard's single-packet handshake design principle. The fragmentation penalty of the remaining variants is structurally unavoidable given the sizes of current NIST-standardized KEM parameters.

Two approaches currently exist for reducing message sizes further. The first is to exploit algebraic structure within the KEM to compress multiple operations into a single ciphertext, as demonstrated in the Rebar paper [11], at the cost of departing from the standardized primitive. The second is to reduce the security level, accepting a smaller margin against future cryptanalysis.

The IPv6 budget of 1232 bytes is even tighter than IPv4, and none of the KEM variants fit within it in the initiation direction. This is a significant concern for future deployments, particularly in environments where IPv6 is preferred or mandatory.

8.3 Full Post-Quantum Authentication

The pure-KEM mode is the only variant in this thesis that achieves full post-quantum authentication. In this mode, both the session key material and the identity commitments are based entirely on KEM operations over post-quantum public keys. A quantum adversary equipped with Shor's algorithm cannot break either component, since no classical operations remain in the handshake. Altough, achieving it comes at a substantial cost in message size. The initiation message must carry more than twice the KEM material that the hybrid mode requires.

A second practical cost is the out-of-band static key provisioning. Each peer must be configured with the KEM static public keys of all its peers before any handshake can take place. As shown in Table 6, this remains manageable at realistic deployment scales, but it is still a 37-fold increase over the 32-byte Curve25519 static key at the very least. Operator tooling, configuration file formats, and key distribution systems would all need to be updated to handle these larger keys.

So, for operators whose primary concern is data confidentiality under the HNDL threat model, the hybrid mode is sufficient. For operators who need to guarantee authentication against a quantum adversary, the pure-KEM mode is required despite its larger messages.

8.4 Relation to Prior Work

8.4.1 PQ-WireGuard

PQ-WireGuard [10] demonstrated that post-quantum key exchange is achievable within WireGuard's packet budget by transmitting only the KEM ephemeral material in-band and distributing the large Classic McEliece static keys out of band. This thesis takes a different approach as ML-KEM's much smaller key and ciphertext sizes make it possible to include all necessary key material directly in the handshake messages without special pre-distribution. The trade-off is that the pure ML-KEM variants exceed the IPv4 budget in the initiation direction, whereas PQ-WireGuard's design does not. However, the per-peer storage cost of ML-KEM is several orders of magnitude lower than that of Classic McEliece, which makes the approach more practical at scale.

8.4.2 Rosenpass

Rosenpass [9] is the most practical deployment option among the related work. It requires no changes to WireGuard's codebase and installs as a sidecar process that runs a separate post-quantum key exchange alongside the classical WireGuard handshake.

The compactness of Rosenpass stems from the same approach as PQ-WireGuard, where the static Classic McEliece key is never transmitted during the handshake, sidestepping the MTU problem entirely while carrying the same large per-peer storage overhead. Its main limitation is that it only provides post-quantum forward secrecy, as authentication still relies on the classical WireGuard handshake, leaving it vulnerable to a quantum adversary running Shor's algorithm. The pure-KEM mode in this thesis addresses this gap by achieving full post-quantum authentication, at the cost of a more invasive code change.

8.4.3 Rebar

Rebar [11] is the most direct point of comparison for the pure-KEM variants in this thesis, since both use ML-KEM as their primary building block.

Rebar's formally proven security model that covers entity authentication, denial-of-service resistance, and identity hiding in the computational model is a significant advantage that the informal security arguments of this thesis cannot match. A future production-grade post-quantum WireGuard would likely need to follow the Rebar security model or provide an equivalent formal analysis. Rebar's use of the RKEM construction also allows both handshake messages to fit within the IPv6 budget, a goal that the straightforward ML-KEM integration in this thesis does not achieve.

8.5 Ethics and Sustainability

The transition to post-quantum cryptography is fundamentally about protecting the privacy and security of individuals and organizations against future threats. The "harvest now, decrypt later" model means that inaction today has real consequences for people whose communications are being collected and stored. At the same time, deploying new cryptographic primitives that have not yet received the same

decades of scrutiny as classical schemes carries its own risk. The hybrid approach advocated for in this thesis is therefore a way forward: by ensuring that a failure in the post-quantum component does not leave users worse off than before, it provides a safer path through the transition.

From a sustainability perspective, the computational overhead introduced by post-quantum algorithms has energy implications at scale. The modest overhead of ML-KEM observed in this thesis is encouraging in that regard, while the substantially higher cost of HQC serves as a reminder that algorithm selection for post-quantum migration is not only a security and performance decision, but an environmental one.

8.6 Future Research

Several open questions emerged during the making of this thesis.

RKEM integration with standardized ML-KEM. Rebar demonstrates that the RKEM construction can reduce handshake message sizes to within the IPv6 budget [11]. An empirical implementation and benchmark of the RKEM construction using the finalized FIPS 203 ML-KEM specification would quantify the latency and complexity cost of this optimization relative to the straightforward ML-KEM integration evaluated here.

Kernel module integration. This thesis is built on `wireguard-go`, which carries TUN interface overhead that is absent in the kernel module. A kernel-level PQC integration using the Linux kernel's native crypto API and liboqs-kernel bindings would produce performance figures more representative of production deployment, and would be necessary for eventual upstream inclusion.

HQC after standardization. HQC's draft standard is expected in early 2026 and its final standard in 2027 [27]. The finalization process may produce optimized implementations that reduce the algorithm's computational overhead. It would be worth re-evaluating HQC once such implementations are available, particularly in the context of emerging KEM designs that might achieve smaller ciphertext sizes for equivalent security levels.

Post-quantum authentication migration paths. The hybrid and pure-KEM modes represent two different points on the migration timeline, but the transition between them in a deployed network would require operators to distribute new static KEM public keys to all peers before switching from hybrid to pure-KEM mode. Research into key distribution protocols, backwards-compatible negotiation mechanisms, and migration tooling would be needed to make this transition practical at scale.

9 Conclusions

This thesis set out to answer how NIST-approved post-quantum key encapsulation mechanisms, specifically ML-KEM and HQC, affect the performance, resource usage, and practical deployability of a post-quantum secure WireGuard handshake. And after much research, the answer is quite clear.

ML-KEM integrates well.

The hybrid ML-KEM-768 variant is the most immediately deployable result of this work. It augments the existing Curve25519 exchange with a single KEM operation, fits within a single IPv4 packet in both message directions, and adds approximately 22% to the handshake time on tested hardware, which, when talking milliseconds, is a small price to pay for the security benefits it brings. Furthermore, the hybrid implementation produces per-peer storage requirements that are manageable at realistic deployment scales, resolving the principal practical limitation of PQ-WireGuard's Classic McEliece approach.

HQC is not suitable for WireGuard in its current form.

The fundamental problem is not computational performance but message size. HQC ciphertexts are large enough to force up to 30 IP fragments per handshake, producing mean latencies of 200 to 670 ms that place every HQC variant far outside the performance criteria that makes WireGuard valuable. This may change as NIST's standardization process produces more optimized implementations, but in its current form HQC is incompatible with WireGuard's MTU-constrained design.

The MTU constraint remains the defining challenge.

Only Hybrid-MLKEM-768 fully satisfies WireGuard's single-packet design assumption, while the other KEM variants required fragmentation. Prior work by Hashimoto et al. demonstrates that the RKEM construction can fit two ML-KEM operations within the IPv6 budget by exploiting algebraic structure internal to the algorithm. Until such optimizations are standardized, the MTU constraint remains a hard ceiling that algorithm selection alone cannot overcome.

The HNDL threat is addressable today.

The regulatory deadlines under CNSA 2.0 and the European Commission: mandatory PQC transition by 2026, full transition by 2030, are within reach using the hybrid approach described in this thesis. Operators can deploy the hybrid mode today to gain confidentiality protection against future quantum adversaries without replacing existing static key infrastructure.

The urgency of the post-quantum transition is real. Traffic protected by classical key exchange today is already at risk under the "harvest now, decrypt later" model, and the window for an orderly migration is narrowing. The main remaining challenge is not computational but practical: deploying these changes across a fragmented ecosystem of implementations and VPN services.

Hybrid-ML-KEM-768 threads this needle today. Pure ML-KEM, and eventually a fully standardized post-quantum WireGuard, represent the path forward.

References

[1] M. Swayne. "Quantum Computing Roadmaps: A Look at the Maps and Predictions of Major Quantum Players." Accessed: 2026-03-10, The Quantum Insider. [Online]. Available: https://thequantuminsider.com/2025/05/16/quantum-computing-roadmaps-a-look-at-the-maps-and-predictions-of-major-quantum-players/

[2] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Review*, vol. 41, no. 2, pp. 303–332, 1999. DOI: 10.1137/S0036144598347011 eprint: https://doi.org/10.1137/S0036144598347011. [Online]. Available: https://doi.org/10.1137/S0036144598347011

[3] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96, Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 212–219, ISBN: 0897917855. DOI: 10.1145/237814.237866 [Online]. Available: https://doi.org/10.1145/237814.237866

[4] J. Mascelli and M. Rodden, ""Harvest Now Decrypt Later": Examining Post-Quantum Cryptography and the Data Privacy Risks for Distributed Ledger Networks," Board of Governors of the Federal Reserve System, Washington, DC, Tech. Rep. 2025-093, 2025. DOI: 10.17016/FEDS.2025.093 [Online]. Available: https://doi.org/10.17016/FEDS.2025.093

[5] National Institute of Standards and Technology, *Post-Quantum Cryptography (PQC) Standardization Process*, https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization, [Online; accessed 19-February-2026], 2025.

[6] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," in *NIST, Tech. Rep.*, 2024. DOI: 10.6028/NIST.FIPS.203 [Online]. Available: https://doi.org/10.6028/NIST.FIPS.203

[7] C. Aguilar-Melchor et al., "Hamming Quasi-Cyclic (HQC) Specifications," HQC Team, Tech. Rep., 2025, Version dated 2025-08-22. [Online]. Available: https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf

[8] J. A. Donenfeld, "Wireguard: Next generation kernel network tunnel," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, The Internet Society, 2017. DOI: 10.14722/ndss.2017.23160 [Online]. Available: https://www.wireguard.com/papers/wireguard.pdf

[9] K. Varner, B. Lipp, L. Schmidt, P. Dua, and W. Zaeske, "Rosenpass," 2024. [Online]. Available: https://rosenpass.eu/whitepaper.pdf

[10] A. Hülsing, K.-C. Ning, P. Schwabe, F. J. Weber, and P. R. Zimmermann, "Post-quantum wireguard," in *2021 IEEE Symposium on Security and Privacy (SP)*, 2021, pp. 304–321. DOI: 10.1109/SP40001.2021.00030

[11] K. Hashimoto, S. Katsumata, G. Niot, and T. Wiggers, "Revisiting PQ WireGuard: A comprehensive security analysis with a new design using Reinforced KEMs," in *IEEE Security & Privacy 2026*, 2026. [Online]. Available: https://thomwiggers.nl/publications/pq-wireguard/

[12] A. C. Yao, "Theory and application of trapdoor functions," in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982, pp. 80–91. DOI: 10.1109/SFCS.1982.45

[13] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978. DOI: 10.1145/359340.359342

[14] W. Diffie and M. Hellman, "New directions in cryptography," *IEEE Transactions on Information Theory*, vol. 22, no. 6, pp. 644–654, 1976. DOI: 10.1109/TIT.1976.1055638

[15] E. Barker, "Recommendation for Key Management: Part 1 – General," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. 800-57 Part 1 Rev. 5, May 2020. DOI: 10.6028/NIST.SP.800-57pt1r5 [Online]. Available: https://doi.org/10.6028/NIST.SP.800-57pt1r5

[16] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, "Transition to Post-Quantum Cryptography Standards," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST IR 8547 (Initial Public Draft), Nov. 2024. DOI: 10.6028/NIST.IR.8547.ipd [Online]. Available: https://doi.org/10.6028/NIST.IR.8547.ipd

[17] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum Amplitude Amplification and Estimation," *arXiv preprint arXiv:quant-ph/0005055*, May 2000. [Online]. Available: https://arxiv.org/abs/quant-ph/0005055

[18] J. P. Buhler, H. W. J. Lenstra, and C. Pomerance, "Factoring integers with the number field sieve," in *The Development of the Number Field Sieve*, A. K. Lenstra and H. W. J. Lenstra, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 50–94, ISBN: 978-3-540-47892-8.

[19] The CADO-NFS Development Team, *CADO-NFS, An Implementation of the Number Field Sieve Algorithm*, https://cado-nfs.gitlabpages.inria.fr, Development version, 2024.

[20] F. Boudot, P. Gaudry, A. Guillevic, N. Heninger, E. Thomé, and P. Zimmermann, "Comparing the Difficulty of Factorization and Discrete Logarithm: A 240-Digit Experiment," in *Advances in Cryptology – CRYPTO 2020*, ser. Lecture Notes in Computer Science, vol. 12171, Cham: Springer, 2020, pp. 62–91. DOI: 10.1007/978-3-030-56784-2_3

[21] C. Gidney and M. Ekerå, "How to factor 2048-bit RSA integers in 8 hours using 20 million noisy qubits," *arXiv preprint arXiv:1905.09749*, Apr. 2021, Version 3. DOI: 10.22331/q-2021-04-15-433 [Online]. Available: https://arxiv.org/abs/1905.09749

[22] C. Gidney, "How to factor 2048-bit RSA integers with less than a million noisy qubits," *arXiv preprint arXiv:2505.15917*, May 2025. DOI: 10.48550/arXiv.2505.15917 [Online]. Available: https://arxiv.org/abs/2505.15917

[23] National Security Agency, "Announcing the Commercial National Security Algorithm Suite 2.0," National Security Agency, Tech. Rep. PP-22-1338, Sep. 2022, Cybersecurity Advisory, Version 1.0. [Online]. Available: https://media.defense.gov/2025/May/30/2003728741/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS.PDF

[24] EU PQC Workstream, "A coordinated implementation roadmap for the transition to post-quantum cryptography," European Union, Tech. Rep. Part 1, Version 1.1, Jun. 2025, Published June 11, 2025. [Online]. Available: https://digital-strategy.ec.europa.eu/en/library/coordinated-implementation-roadmap-transition-post-quantum-cryptography

[25] D. Moody, *NIST PQC: The Road Ahead*, Presentation, Cryptographic Technology Group, Accessed: 2026-03-24, National Institute of Standards and Technology, Mar. 2025. [Online]. Available: https://csrc.nist.gov/csrc/media/Presentations/2025/nist-pqc-the-road-ahead/images-media/rwcpqc-march2025-moody.pdf

[26] J. Bos et al., "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," in *Proceedings of the 2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367. DOI: 10.1109/EuroSP.2018.00032

[27] D. Moody et al., "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," National Institute of Standards and Technology, Gaithersburg, MD, Tech. Rep. NIST IR 8545, Mar. 2025. DOI: 10.6028/NIST.IR.8545 [Online]. Available: https://doi.org/10.6028/NIST.IR.8545

[28] R. J. McEliece, "A Public-Key Cryptosystem Based on Algebraic Coding Theory," Jet Propulsion Laboratory, Pasadena, CA, Tech. Rep. 42-44, Feb. 1978.

[29] K. Kwiatkowski, P. Kampanakis, B. Westerbaan, and D. Stebila, "Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3," Internet Engineering Task Force, Internet-Draft draft-ietf-tls-ecdhe-mlkem-04, Feb. 2026, Work in Progress, 12 pp. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/04/

[30] B. Westerbaan. "State of the Post-Quantum Internet in 2025." Accessed: 2026-03-24, Cloudflare. [Online]. Available: https://blog.cloudflare.com/pq-2025/

[31] G. Goy, A. Loiseau, and P. Gaborit, "A New Key Recovery Side-Channel Attack on HQC with Chosen Ciphertext," in *Post-Quantum Cryptography – PQCrypto 2022*, J. H. Cheon and T. Johansson, Eds., ser. Lecture Notes in Computer Science, vol. 13512, Cham: Springer, 2022, pp. 353–371. DOI: 10.1007/978-3-031-17234-2_17

[32] Trevor Perrin, *The Noise Protocol Framework*, https://noiseprotocol.org/noise.html, [Online; accessed 5-March-2026], 2018.

[33] E. S. Alashwali and K. Rasmussen, *What's in a Downgrade? A Taxonomy of Downgrade Attacks in the TLS Protocol and Application Protocols Using TLS*, Cryptology ePrint Archive, Paper 2019/1083, 2019. [Online]. Available: https://eprint.iacr.org/2019/1083

[34] J. A. Donenfeld and K. Milner, *Formal Verification of the WireGuard Protocol*, https://www.wireguard.com/papers/wireguard-formal-verification.pdf, Draft revision, n.d.

[35] B. Dowling and K. G. Paterson, "A Cryptographic Analysis of the WireGuard Protocol," in *Applied Cryptography and Network Security*, B. Preneel and F. Vercauteren, Eds., Cham: Springer International Publishing, 2018, ISBN: 978-3-319-93387-0. DOI: 10.1007/978-3-319-93387-0_1

[36] WireGuard, *wireguard-go: Userspace Go Implementation of WireGuard*, https://github.com/WireGuard/wireguard-go, GitHub repository, accessed 2026-05-09, 2026.

[37] Cloudflare, Inc., *BoringTun: Userspace WireGuard Implementation in Rust*, https://github.com/cloudflare/boringtun, GitHub repository, accessed 2026-03-25, 2026.

[38] Open Quantum Safe, *liboqs-go: Go Wrapper for the Open Quantum Safe liboqs C Library*, https://github.com/open-quantum-safe/liboqs-go, Accessed: 2026-05-09, 2026.

[39] Open Quantum Safe, *Liboqs: An open source c library for quantum-safe cryptographic algorithms*, https://openquantumsafe.org/liboqs/, Accessed: 2026-05-09, 2026.

[40] Intel Corporation, *Intel® Advanced Encryption Standard Instructions (AES-NI)*, https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html, Accessed: 2026-05-11, 2012.

[41] H. Noble and J. Smith, "Ensuring validity and reliability in qualitative research," *Evidence-Based Nursing*, 2025. DOI: 10.1136/ebnurs-2024-104232

[42] P. Runeson and M. Höst, "Guidelines for Conducting and Reporting Case Study Research in Software Engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

[43] B. A. Kitchenham et al., "Preliminary Guidelines for Empirical Research in Software Engineering," *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 721–734, 2002.